

Evolving interpretable decision trees for reinforcement learning

Vinícius G. Costa^{a,1}, Jorge Pérez-Aracil^b, Sancho Salcedo-Sanz^b,
Carlos E. Pedreira^{a,*}

^a Systems Engineering and Computer Science Department, COPPE, Federal University of Rio de Janeiro, Rio de Janeiro, Brazil

^b Department of Signal Processing and Communications, Universidad de Alcalá, Alcalá de Henares, 28805, Madrid, Spain

ARTICLE INFO

Keywords:

Interpretability
Reinforcement learning
Decision trees
Evolutionary-based algorithms
Multi-method ensembles

ABSTRACT

In recent years, reinforcement learning (RL) techniques have achieved great success in many different applications. However, their heavy reliance on complex deep neural networks makes most RL models uninterpretable, limiting their application in domains where trust and security are important. To address this challenge, we propose MENS-DT-RL, an algorithm capable of constructing interpretable models for RL via the evolution of decision tree (DT) models. MENS-DT-RL uses a multi-method ensemble algorithm to evolve univariate DTs, guiding the process with a fitness metric that prioritizes interpretability and consistent high performance. Three different initializations for the MENS-DT-RL are proposed, including the use of Imitation Learning (IL) techniques, and a novel pruning approach that reduces solution size without compromising performance. To evaluate the proposed approach, we compare it with other models from the literature on three benchmark tasks from the OpenAI Gym library, as well as on a fertilization problem inspired by real-world crop management. To the best of our knowledge, the proposed scheme is the first to solve the Lunar Lander benchmark with both interpretability and a high confidence rate (90% of episodes are successful), as well as the first to solve the Mountain Car environment with a tree of only 7 nodes. On the real-world task, the proposed MENS-DT-RL is able to produce solutions with the same quality as deep RL policies, with the added bonus of interpretability. We also analyze the best solutions found by the algorithm and show that they are not only interpretable but also diverse in their behavior, empowering the end user with the choice of which model to apply. Overall, the findings show that the proposed approach is capable of producing high-quality transparent models for RL, achieving interpretability without losing performance.

1. Introduction

In recent years, deep Reinforcement Learning (RL) algorithms have been applied with great success to various tasks, such as robotics [1], resource management [2], recommendation systems [3], and playing Go at superhuman levels [4]. While these results are impressive, deep RL algorithms typically use Deep Neural Networks (DNNs) with complex multi-layer structures and thousands (if not millions) of parameters. For this reason, the resulting models are generally considered to be uninterpretable, which is in line with the popular understanding of DNNs as “black boxes” [5–7].

* Corresponding author.

E-mail address: pedreira@cos.ufrj.br (C.E. Pedreira).

¹ This work was partially done while VGC was in a stay at the Universidad de Alcalá.

The uninterpretability of deep RL models has several drawbacks. The first of those is related to the current great concern about potentially harmful effects that can be caused by artificial intelligent systems: since the “model’s decision-making process” is incomprehensible to humans, it is hard or perhaps impossible to guarantee that it will not execute undesired and potentially harmful actions [8]. Second, uninterpretability complicates troubleshooting, since it is hard to understand why the model chose one action over another [8,9]. Third, there are legal concerns, since transparency and accountability are increasingly being legislated as prerequisites for the deployment of autonomous solutions in real-world environments [10]. Finally, for high-stakes domains where safety is critical (such as healthcare [11,12] and autonomous driving [13]), the lack of transparency of DNN-based approaches may lead to solutions that are impossible to trust and therefore to deploy. These drawbacks have recently led to a great deal of attention being paid to interpretable RL: several recent works and surveys have been dedicated to it [9,10,14,15], and the task has been identified as one of the major challenges for interpretable ML as a whole [8].

An intuitive way to incorporate interpretability into RL is to replace the DNNs with more interpretable models, such as the widely popular decision trees (DTs): rule-based models, usually presented in a flowchart-like structure that consists of logical tests and predictions [16]. Because their inner workings are completely transparent and simulatable by the end user, DTs are often considered the go-to techniques in interpretable ML, sometimes referred to as “white boxes” [17–19]. But while mixing DTs with RL may seem intuitive, there is no obvious way to perform this integration, mainly because traditional DT algorithms require that the entire dataset be available at once, while RL algorithms typically work on an online, sample-by-sample basis. The literature has attempted to reconcile this conflict by introducing several different approaches, ranging from the gradient-based [20] to the greedy [21]. However, most works end up sacrificing either interpretability or model performance, resulting in trees that cannot directly compete with more complex and uninterpretable approaches.

To address this challenge, we propose the “Multi-method ENsemble for Decision Trees in Reinforcement Learning” (MENS-DT-RL), an evolutionary-based algorithm capable of producing interpretable and high-performing DTs for RL tasks. The algorithm works by adapting a multi-method ensemble algorithm (CRO-SL [22,23]) that has been successful at optimization problems both in machine learning [24,25] and engineering [26,27]. We extend the considered multi-method ensemble with a set of genetic operators capable of handling DT models, as well as with a fitness metric that rewards trees with consistent behaviors and smaller sizes. Although the basic MENS-DT-RL presents high-quality results for simpler tasks, its true value appears when it is initialized with good solutions from an Imitation Learning (IL) approach, especially when these solutions are pruned using a novel technique that adapts traditional post-pruning algorithms to the RL environment. The main contributions of this work can be summarized as follows:

- We propose a novel evolutionary algorithm for interpretable RL, based on a multi-method ensemble optimization algorithm.
- We develop a method capable of pruning DTs in RL environments, inspired by traditional DT pruning methods from supervised learning.
- We evaluate the proposed algorithm over three popular benchmarks from the OpenAI Gym environment [28] and a crop management task based on real-world fertilization problems [29]. To the best of our knowledge, this algorithm surpasses the state-of-the-art with regards to interpretable DTs for RL, finding the new smallest DT capable of solving the Mountain Car environment with a perfect success rate, being the first to solve the Lunar Lander task with a high success rate ($\geq 90\%$ of successful trials), and surpassing all previous expert policies for the real-world fertilization task.
- We demonstrate not only that the resulting solutions are interpretable, but also that they are diverse, and therefore empower the user with the ability to choose between distinct behaviors for the desired task.

The rest of the paper is organized as follows: Section 2 discusses the necessary knowledge and offers a review of the related works in the literature. Section 3 describes the MENS-DT-RL algorithm and its initialization procedures. Section 4 contains the experiments and discussion, and Section 5 presents our concluding remarks.

2. Background and related works

With the goal of making the paper self-contained, in this section we briefly cover the topics of Reinforcement Learning and Imitation Learning, as well as briefly introduce the multi-method ensemble algorithm that serves as basis for the proposed MENS-DT-RL technique. At the end of the section, we offer a review of the related works.

2.1. Reinforcement learning

Reinforcement Learning (RL) [30] is an area of machine learning that aims to create intelligent agents that observe the current state of the world, process it, and decide what is the best possible action to take according to a previously defined goal. Traditionally, there are three key components to an RL problem: *states*, which contain the environmental data that the agent can observe at a given moment; *actions*, which encapsulate the ways in which the agent can affect the environment; and *rewards*, which are the feedback that the agent receives after taking a certain action in a certain state. In an ideal learning process, the agent starts by taking random actions and receiving poor rewards, but eventually learns how to use the current state to figure out the best action to take, thus learning a “behavior function” that maximizes the prospective reward. Such functions are called *policies*, and are usually denoted by $\pi : S \rightarrow A$.

Using the immediate reward is typically not enough to learn good policies: often the consequences of an action are felt only several timesteps in the future, which complicates the issue of learning which actions were well taken and which were not. In the

most extreme cases, a reward is given only after a terminal state is reached (i.e., at the end of so-called *episodes*, finite sequences that begin in an initial state and end in a terminal one). The traditional example is a chess game, in which the agent collects a reward of +1 if it wins, -1 if it loses, and 0 if it draws; until an outcome is reached, the reward is continuously 0, which severely limits the feedback the agent receives throughout the learning process.

These and other problems can be addressed to some extent by techniques such as the traditional Q -learning algorithm [31]. In this algorithm, the agent maintains a Q -table, which is a table where each entry contains a Q -value (i.e. the expected cumulative reward of taking a given action in a given state). Through iterative updates based on observed rewards and transitions between states, Q -learning modifies these Q -values to approximate the optimal policy, continuing until either the Q -table converges or the agent reaches a desired level of performance. Although Q -learning is a cornerstone of RL literature and spawned several extensions [32–34], there are other fronts to the field that do not build directly upon it, such as Imitation Learning.

2.1.1. Imitation learning

Imitation Learning (IL) [35] is a sub-field of RL in which the goal is not to train an agent from scratch, but instead to imitate another agent called an *expert*. These techniques find great use in tasks where it is easier to demonstrate the correct behavior than to implicitly define it through a reward function, such as in autonomous driving [36], human-like locomotion [37] and object manipulation [38], among others. The expert is usually taken to be a human demonstrator, but most IL techniques can also be applied when the target of imitation is another RL agent.

The most basic IL approach, Behavioral Cloning, simply collects a dataset of demonstrations $D = \{(s, a)\}$ and fits a supervised learning algorithm to the data [39]. The result is a model capable of predicting an action given an input state, and that therefore can be used in much the same way as an RL agent. Although intuitive, this naive approach has been shown to be susceptible to “covariate shift”: a condition in which the imperfect imitator, due to its imperfection, sometimes takes non-expert actions and ends up in states that the expert would not visit. Since the expert would not visit these states, they are not included in the dataset D used to train the imitator, which leads it to take even more non-expert actions and end up in even more unfamiliar states. Because this effect often leads to bad results, more sophisticated algorithms were proposed, such as DAgger [40].

In DAgger, covariate shift is tackled by extending the dataset D with these distinct situations brought by the imitator’s imperfection. At every i -th iteration of DAgger, a new imitator π_i is produced by fitting a supervised learning model on the dataset D , just like in Behavioral Cloning. Then, the dataset is expanded: first, the imitator π_i freely interacts with the environment and visits a set of states, which are stored by the algorithm. Next, this set of states is passed on to the expert agent, who labels them with the expert action it would take in each state. The result is a dataset D_i that contains imitator states but expert actions, which is then aggregated unto D for the next $(i + 1)$ -th iteration. Through this iterative approach, DAgger finds much more success than the simpler Behavioral Cloning, since covariate shift is addressed by having D contain a wider array of states. The key disadvantage of DAgger, however, is that it requires continuous access to an expert – which may be costly or impossible, depending on the domain.

Since IL effectively reduces RL problems into a supervised learning framework, DTs for RL can be easily trained by using off-the-shelf traditional DT algorithms as imitators. However, there is no guarantee that these trees will be interpretable, since in trying to faithfully imitate the experts they can end up being unreasonably large. Therefore, more specialized solutions are necessary, such as the ones proposed in this paper.

2.2. Multi-method ensemble algorithm for optimization

In optimization problems, an *ensemble* method for optimization refers to an algorithm that combines different types of alternative methods, search strategies or operators, to obtain high-quality solutions [41]. The application of ensemble approaches to solve optimization problems has been very important in the last few years, due to the good results obtained by these combinations of techniques in hard optimization problems and real applications. Following [41], there are different types of ensemble approaches, but the most used are those trying to obtain an optimal combination of different types of search strategies or operators within a single algorithm (multi-method ensembles). The main idea behind these ensemble approaches is to exploit the capacity of different search methods/operators by combining them in several possible ways, in order to improve the search ability of the final approach in optimization problems.

Different types of multi-method ensembles for optimization have been described in the literature. An example of multi-method competitive ensemble is [42], in which different operators are applied in a single evolutionary algorithm-based ensemble. An approach with a similar idea was proposed in [43]. Multi-method approaches have also been applied to improve the performance of meta-heuristics in multi-objective optimization problems [44]. There are also multi-method algorithms which work on different sub-populations such as [45], and ensembles of multi-strategy algorithms based on algorithms such as Differential Evolution [46,47].

In this paper we consider the CRO-SL multi-method ensemble [22], an algorithm that has obtained excellent results in hard optimization problems in the past [48,25,49,50]. Specifically, we apply the probabilistic-dynamic version of the algorithm [51]. Details on the implementation of the multi-method ensemble, as well as a Python implementation of the algorithm, can be found in [51].

2.3. Related works

In this section, we review those works that similarly tackle the issue of interpretability in RL. Given the vastness of the field, a comprehensive review is outside the scope of the current paper; instead, we focus on contributions that also tackle interpretable RL

through the DT angle. However, given that DTs are not the only representation that has been explored in this literature, we start out by taking a non-exhaustive approach to position DTs within the larger context of policy structures that have been previously explored in interpretable RL.

Formulas are a common representation in interpretable RL. In these models, each action is expressed as a function of the attributes in the state s ; for example, in an air conditioner task, the temperature of the unit could be represented as $f(x) = 20.175 + 0.05temp - 0.0045temp^2$, where *temp* is the current temperature in the room. There is no well-defined way to obtain formulas of this kind: for instance, [52] used genetic programming techniques to generate formulas from a historical dataset of environmental state-action trajectories, while [53] used a novel method called evolutionary feature synthesis to create complex features from state attributes, which were then assembled into functions through linear models. In general, these formula-based solutions are not readily comparable to DT agents: although formulas can be clear, concise, and particularly insightful when the underlying model lends itself well to mathematical modeling, DTs are easier to visualize and interpret without number crunching, a property that may be more desirable to the end user. One model is not more interpretable than the other – each has its own flavor of interpretability.

Another structure used in interpretable RL is that of programmatic policies. These models, which can be seen as a generalization of the formulas described above, use a mix of Boolean, algebraic, and conditional operators to create a system that closely resembles human-made computer programs. In the previous air conditioning task, an example agent of this type would be “*if (temperature > 30) and (humidity > 20%) then set air conditioner to (41.66 – 0.66 × temperature) else turn it off*”. As with formulas, a wide variety of techniques have been used in the past to obtain these models: [54] built them by searching the latent space generated by variational autoencoders, [55] did the same by combining Bayesian optimization with imitation learning, and [56] derived a Bayesian inference algorithm to optimize agents from few-shot datasets. Again, comparing these types of solutions to DTs is difficult: programmatic policies are more flexible, can handle more complex logic, and are closer to human computer programs, which tend to be interpretable. However, due to their flexibility, these machine-generated programs can also quickly become unclear and difficult to understand, a problem that DTs do not face with the same intensity due to their more constrained and visual structure.

Fuzzy control systems, which consist of a series of fuzzy IF-THEN rules, are a third type of representation used in several works. An example for an air conditioner control problem would be “*IF temperature is hot THEN turn A/C to cold*”, where “hot” and “cold” are defined by fuzzy membership functions and can take a predefined range of values (in contrast to a “crisp” logical test like “IF temperature > 35 THEN turn A/C on”, which would do nothing if the temperature was 34.9 degrees). Although fuzzy control systems were originally proposed in the '70s [57], they were usually defined by hand rather than learned from data, leaving a gap which researchers tried to fill with a wide range of techniques. [58] used genetic algorithms to evolve individuals as rules, combining them to produce a complete solution in a framework they called “symbiotic.” [59] mixed traditional RL algorithms with modern fuzzy techniques to solve the problem in an online manner, avoiding the need to relearn as the environment changes. [60] used the Particle Swarm Optimization (PSO) metaheuristic to generate these rules from a batch dataset, similar to their posterior work on evolving formulas [52]. Compared to DT models, fuzzy rule sets are often more powerful, have conditions that may be easier to understand (due to the use of concepts as thresholds), and have fewer rules than a DT would have if converted to the same rule representation. However, DTs have a clearer hierarchical structure, can be represented visually, and do not require external analysis of membership function graphs in order to be mentally simulated, which makes them more straightforward to interpret. As with the other models discussed previously, the choice ultimately comes down to personal taste, as each representation has its own approach to interpretability.

Next, we move on to a more in-depth discussion of works that tackle interpretable RL through the use of DTs. In this line of research, four different approaches can be discerned: greedy, imitation-based, gradient-based and evolutionary learning.

The *greedy* approach attempts to build the DTs iteratively from scratch, similarly to traditional greedy DT algorithms like CART [61] and C4.5 [62]. The first contribution in this line of work was proposed by Pyeatt and Howe [63]: DTs were used to predict the Q -value of every (state, action) observation so that each leaf contains a history of the ΔQ updates applied to it. If this sample history reaches a sufficiently high variance, the leaf is split into two, since it supposedly contains more than one relevant region of the input space. The original paper does not report tree sizes, but given that the procedure is strictly additive (the tree only increases in size), it seems safe to assume that the produced trees are not particularly small, which can severely impact the interpretability of the final model. A related approach can be found in U-Trees [64] and its extension, Continuous U-Trees [65]. Both these algorithms also store online information in the leaves and split them when a condition is fulfilled, but in their case, the leaf-action mapping is defined by solving the DT's corresponding Markov Decision Process, while the splitting criterion is either a Kolmogorov-Smirnov test or a check for Markov property violation. These two algorithms suffer from the same issue as the proposal by Pyeatt and Howe, in the sense that they are strictly additive and tree size is not reported. This issue also impacts Fitted Q -Iteration (FQI; [66]), a more popular approach to mixing DTs with RL. At each iteration, a new regression tree is trained on a dataset composed of (state, action) pairs as inputs and Q -values as outputs, which in theory gradually approximates the optimal policy. While FQI can produce DT's for RL, the final model cannot be easily interpreted – not only because of the additive issue mentioned above, but also because each leaf represents a single Q -value (instead of an action). This means that in order to choose an action with an FQI tree, it is necessary to simulate the tree for every possible action and remember which action had the highest Q -value (assuming actions are taken greedily). Naturally, this process is much less interpretable than simply having actions for leaves and simulating the tree once, which is the approach taken in this paper.

Another branch of contributions involves *imitation-based* approaches, which borrow ideas from the IL literature to train DTs based on uninterpretable models. In [67], authors proposed an algorithm called VIPER, which extends the widely-used IL algorithm DAGger [40] and differentiates itself by learning not the actions but instead the Q -values themselves. The authors show that by

representing the policy as a DT structure, it is possible to verify logically that certain states are never reached, which is key for achieving trustworthy models. However, since the authors were not focused on interpretability, it is not clear if VIPER is capable of producing small and interpretable trees (indeed, the original paper reports trees with close to 700 nodes). Another approach in the same line is [68], which does not employ a traditional IL algorithm, but instead builds upon a method proposed by [69] to distil a DNN into the structure of a DT. Although the final models obtained high average reward in a challenging task, the authors employed a variant of DTs called “soft trees”, which replaces the traditional univariate logical tests in the inner nodes with a full linear combination of parameters that are then passed onto a non-linear function (akin to a DNN’s neuron). For this reason, it is debatable if the proposed solution achieves traditional notions of interpretability.

The *gradient-based* approach trains DTs for RL by replacing the traditional greedy approach with online, gradient-based algorithms. This can be seen as part of a broader movement in DT research to keep the interpretability of a DT while reaping the benefits of DNNs’ flexible training algorithms, creating hybrid models that have been called gradient-based trees [16] or neural trees [70]. Following this idea, [71] proposed Linear Model U-Trees (LMUT), an extension of the greedy Continuous U-Trees that employs linear models at the leaves and trains them via stochastic gradient descent. Although the model obtains some degree of success, the produced trees are not interpretable in a traditional sense, since each leaf contains not a single comprehensible action but an entire linear model itself. A final gradient-based method was proposed by [20]. The authors built upon an earlier algorithm by [72], which used stochastic gradient descent to train soft trees; by adapting this algorithm, training DTs for RL becomes straightforward. However, as was pointed previously when discussing [68,69], this “soft tree” approach arguably lacks interpretability, and although the authors provided a solution to make the model more interpretable by converting the soft splits into univariate ones, this process resulted in some loss of performance. Thus, better trees may be found by aiming directly for univariate solutions from the start.

Finally, *evolutionary* approaches employ evolutionary algorithms (EAs) to train DTs for RL tasks. In [73], the authors use a Grammatical Evolution approach to evolve the tree’s inner splits, while the leaves are determined by running Q-learning for a low number of episodes. Although this hybrid procedure reduces the number of parameters that need to be evolved by the EA (which usually has the positive effect of accelerating the algorithm), it can also bias the results towards trees that are larger than necessary, since it has been shown in the literature that the tree size needed to obtain an optimal policy is sometimes larger than the tree size needed to represent it [64]. Furthermore, the proposed approach is not able to obtain small univariate trees for more sophisticated environments such as Lunar Lander, which the algorithms proposed here are able to. Finally, [74] uses EAs to evolve a structure known as non-linear DTs, in which the inner splits are non-linear inequalities such as $w_1x_1^{-3} + w_2x_2^3 - w_3 \leq 0$. This augments the representation capacity of DTs and therefore reduces their size, but it also limits the traditional notions of interpretability for which DTs are known.

3. Multi-method ensemble for decision trees in reinforcement learning (MENS-DT-RL)

In this section we describe the MENS-DT-RL algorithm, which is applied to evolve interpretable DTs for RL tasks. In essence, the MENS-DT-RL closely follows the base structure of the CRO-SL algorithm described in [51], such that each individual in the ensemble corresponds to a different tree agent, and each mutation operator applies a distinct modification to the tree structure (e.g. removing a node, adding a node, etc). Then, when an individual fitness must be calculated, the algorithm takes the corresponding tree and runs it over a set of N episodes, using the total reward of each episode to calculate the individual’s fitness through a certain function. The overall approach is illustrated in Fig. 1, and the fitness evaluation is illustrated in Fig. 2. In what follows, we provide a detailed look at the components of the MENS-DT-RL.

The first component is the *solution representation*. In the original multi-method ensemble in [51], solutions are represented as a vector or matrix of real numbers, which are then modified in order to produce a good solution. This allows for the usage of numerical optimization algorithms as operators, such as Differential Evolution and Harmony Search (both of which are already implemented in the open-source implementation from [51]). However, although matrix representations of DTs have been proposed in the past [75–77], in this work we employ a more straightforward and intuitive representation of each solution as a traditional univariate DT. Since numerical operators are not applicable under this representation, new ones were proposed for MENS-DT-RL, allowing an input tree to be transformed into any other possible univariate tree. The operators are illustrated in Fig. 3, and can be described as follows:

1. *Replace with child*: select a random inner node and uniformly select either its left or right subtree. Replace the selected inner node with its selected subtree.
2. *Truncate*: select a random inner node and uniformly select either its left or right subtree. Replace the selected subtree with a randomly generated leaf.
3. *Insert inner node*: select a random inner node from the tree, and between it and its parent, insert a new random inner node. Assign the selected inner node as the left or right child of the newly generated inner node with 50-50 probability.
4. *Expand leaf*: select a random leaf and turn it into an inner node with two random leaves for children.
5. *Reset split*: select a random inner node. Replace its attribute with a randomly selected one, and replace its threshold with a uniform value between -1 and 1.
6. *Modify threshold*: select a random inner node and apply to its threshold a Gaussian perturbation of mean 0 and standard deviation of 0.1.
7. *Modify leaf (discrete)*: select a random leaf and replace its action with a new randomly selected one.
8. *Modify leaf (continuous)*: select a random leaf and replace its action with a random value in the range of possible continuous actions.

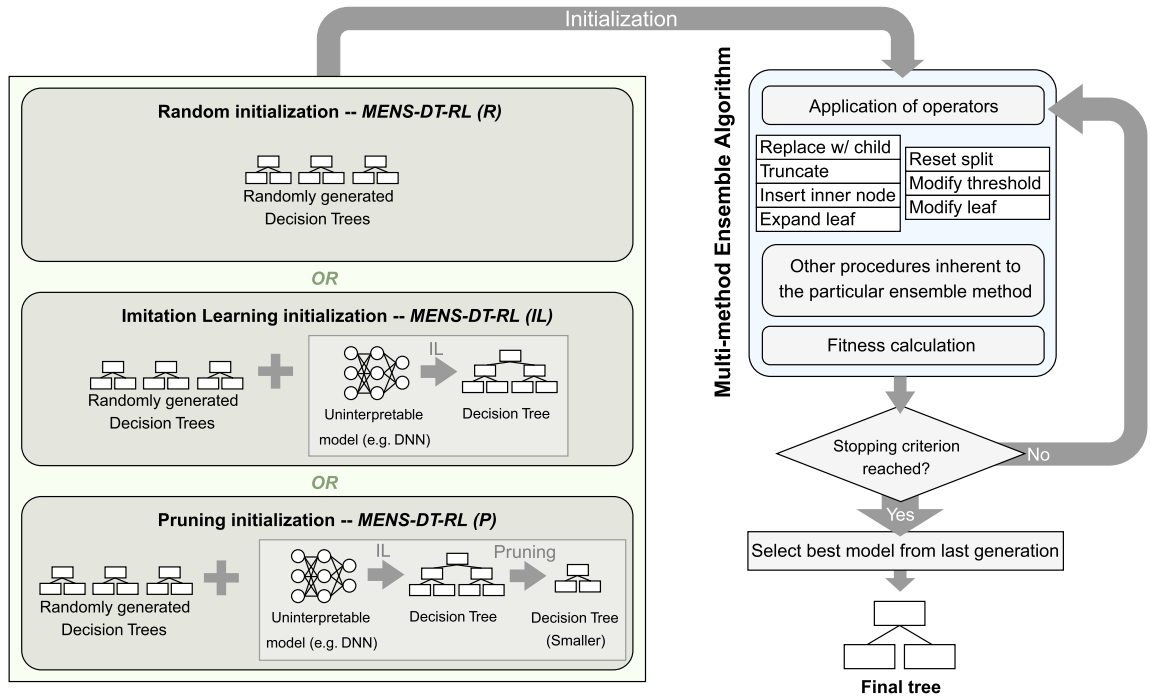


Fig. 1. An illustration of the overall MENS-DT-RL algorithm. First, an initialization procedure is selected (either random trees, random trees and trees obtained via Imitation Learning, or random trees and trees obtained via Imitation Learning that were pruned using Reward Pruning). Then, the initial population is passed onto a multi-method ensemble algorithm in order to produce an improved solution, which is ideally a small and high-performing tree agent. In this paper, the particular multi-method ensemble used is the Probabilistic CRO-SL [51].

The second key component in the MENS-DT-RL algorithm is the *fitness*. As is usual in evolutionary computation, each individual is evaluated in accordance with a fitness metric that corresponds to the quality of each solution. Since we are dealing with RL problems, the most natural metric for performance would be the average cumulative reward, since it is the one most often used to compare different algorithms. However, using only the average reward might bias the algorithm towards models that are needlessly large: for instance, there might be a situation where a tree is half the size of its predecessor, but has only a slightly lower average reward – an algorithm that focuses only on reward would ignore this more interpretable solution. For this reason, it might be interesting to include tree size in the fitness function, in addition to the average reward. It must be said, however, that a lower tree size does not always imply a more interpretable tree: for instance, end-users might find a larger tree more interpretable than a smaller one if the splits of the larger one are closer to how the user thinks about the problem [78]; in addition, trees with multivariate functions in their splits are usually smaller than their univariate counterparts, but not exactly more interpretable [16]. Indeed, it is impossible to express interpretability with a single metric, given the subjectivity of the topic. However, since a tree with 5 nodes is almost guaranteed to be more interpretable than a tree with 100 nodes that tackles the same task, we opt to use the number of nodes as a proxy for interpretability, even though it is an imperfect one.

Yet, a myopic focus on reward does not only leave out interpretability: it may also bias the algorithm toward models that are inconsistent. A tree with an average reward of 99 and a standard deviation of 20 will be considered inferior to a tree with an average reward of 100 but a standard deviation of 50, while in practice they should be considered at least on an equal level. Given these considerations, the MENS-DT-RL algorithm attempts to build more consistent and interpretable models by using the following fitness metric:

$$fitness(M) = \frac{1}{N} \sum_{i=1}^N G_0^{(M,i)} - \sqrt{\frac{1}{N} \sum_{j=1}^N \left(G_0^{(M,j)} - \frac{1}{N} \sum_{i=1}^N G_0^{(M,i)} \right)^2} - \alpha ||M|| \quad (1)$$

Where M is a DT, $||M||$ is the number of nodes in the tree, N is the number of episodes runs to evaluate fitness, $G_0^{(M,i)}$ is the undiscounted sum of all rewards received in the i -th episode of the tree M (also called “total reward” or simply “reward”), and α is a regularization parameter that penalizes larger tree sizes. From another perspective, this measure is effectively the average reward minus the standard deviation of rewards minus the tree size weighted by α . Including these two components leads to the acceptance of more consistent models that are also more aware of the tradeoffs between performance and size. An ablation study investigating the effects of removing these two components is presented in Appendix A.

A third key aspect of the MENS-DT-RL algorithm is *attribute normalization*. At each step in the RL environment, the attributes for whose minimum and maximum values are known are normalized to a $[-1, 1]$ range, otherwise, they remain unaltered. This step

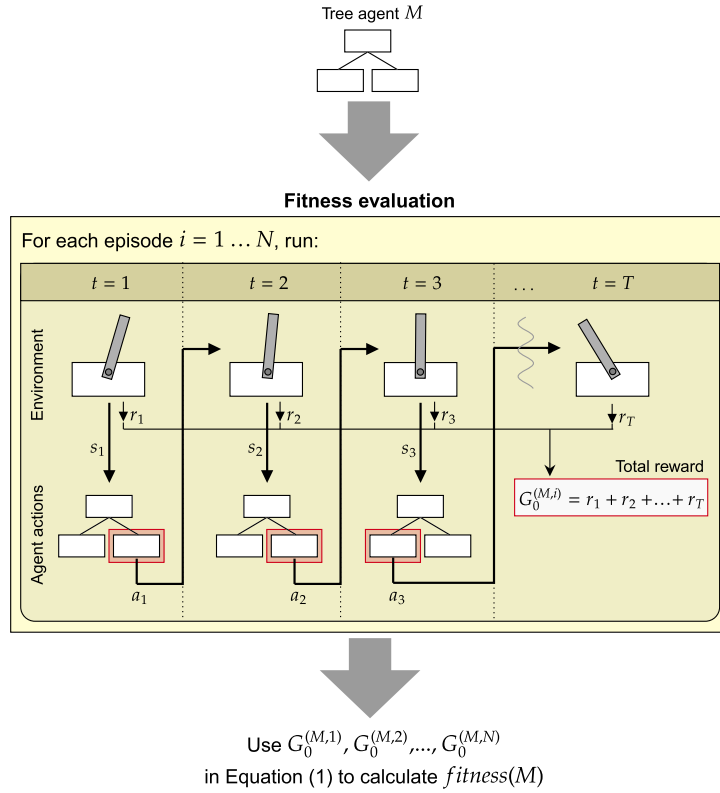


Fig. 2. An illustration of the fitness evaluation procedure. In this example, a tree agent of depth 1 is used on the Cartpole environment. The specifics of how to aggregate $G_0^{(T,i)}$ in order to compute fitness are described in Equation (1).

was empirically observed to greatly improve the quality of obtained solutions since this is the range in which the “modify split” and “reset split” operators work; if this is not included, the algorithm has trouble applying fine-tuning changes to the thresholds in the splits, which end up being either too small to be noticed or too large and catastrophic. A similar normalization was employed in the multivariate DTs of [73].

The final component of the MENS-DT-RL is its *initialization*. The most straightforward way to initialize any evolutionary algorithm is randomly, however, it is known that jumpstarting them with good solutions can greatly enhance the final results [79]. In this work, we experiment with three different initializations for the MENS-DT-RL, which make up its three main configurations:

- **MENS-DT-RL (R):** random initialization. A depth d is selected and a complete tree with this depth is created by uniformly selecting attributes, thresholds, and action leaves.
- **MENS-DT-RL (IL):** IL initialization. CART is employed under the DAgger framework to obtain a tree that imitates a high-performing DNN trained on the task; this tree is then added to the initial population for the MENS-DT-RL.
- **MENS-DT-RL (P):** pruning initialization. This approach takes the previous IL tree and reduces its size without decreasing its performance, in a novel procedure we propose called Reward Pruning. This should provide the MENS-DT-RL with a better initial solution in terms of fitness.

Next, we describe these two last initializations in more detail. The *IL initialization* starts by assuming the existence of an uninterpretable model (called an “expert”) that ideally has good performance on the task. Although any model would suffice, we argue that DNNs are particularly fit for the job: they already have widespread usage across the RL community and there are several resources dedicated to facilitating their training [80], which as a whole greatly alleviates the complexity of the requirement. With the expert in hands, this initialization distils it into a DT using an IL technique: this paper specifically uses the DAgger algorithm [40] to distil the expert into a CART tree model [61], with both DAgger and CART being chosen because they are widely used in their own fields and have a long track record of good-quality results.

An important decision in this phase is the regularization parameter of CART (i.e. the cost-complexity coefficient). If this parameter is too low, the resulting tree will be close to the expert but may have an excessive number of nodes, needlessly making the initial fitness worse. On the other hand, if the regularization parameter is too high, the resulting tree will be overly simplified and badly performing, leading to poor results. It is crucial to find a trade-off that maximizes fitness by finding solutions with high performance but low size. In this work, we define this parameter with a heuristic similar to the elbow method of unsupervised learning: namely, to increase the regularization until the model’s performance is negatively affected. Crucially, the algorithm does not compromise

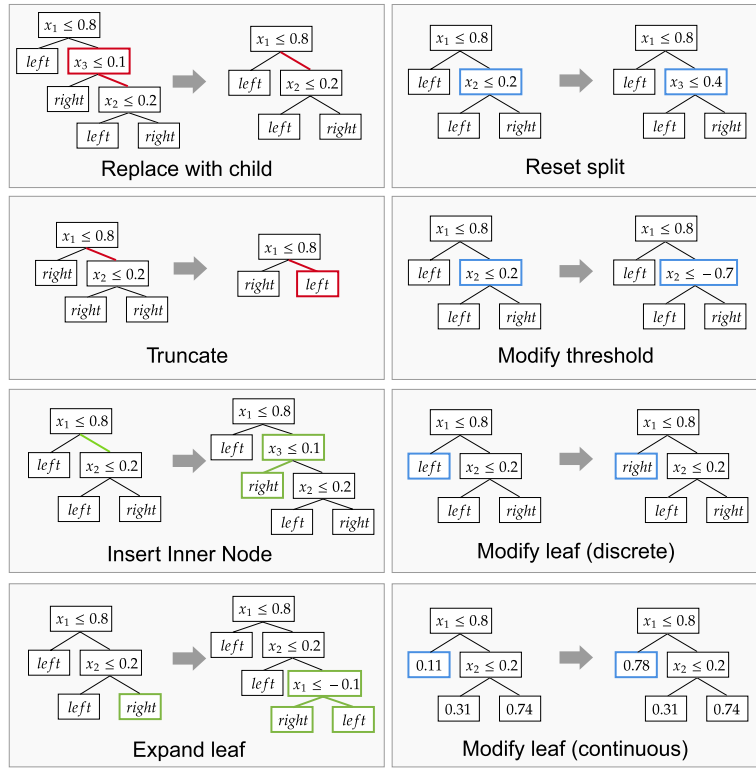


Fig. 3. Operators employed in MENS-DT-RL. In these examples, “left” and “right” are two actions to be taken (the agent moving either left or right).

performance at this stage, as this is already done by the pruning initialization. Further note that, in theory, there is nothing to prevent either Dagger or CART from producing small trees of high quality at this step, but as will be seen in section 4.2, this usually does not happen in practice.

Finally, we describe the *pruning initialization*, which aims to increase the fitness of the initial IL solutions by reducing their size without compromising their performance. The overall approach, which we call Reward Pruning, can be seen as an RL extension of post-pruning algorithms from the DT literature (e.g. CART’s cost-complexity pruning and C4.5’s error-based pruning). However, while these post-pruning algorithms were proposed in a supervised learning environment where the goal was to reduce overfitting, our goal is instead to reduce tree size and increase interpretability, with any improvement in performance being somewhat incidental. The technique is described in Algorithm 1.

Algorithm 1 Reward Pruning.

Require: Expert agent π^*
 //Imitation phase
 $M \leftarrow$ Apply Dagger to imitate π^* with CART

//Pruning phase
 $\text{fitness}(\cdot) \leftarrow$ Defined in Equation (1)
 $\text{SR}(\cdot) \leftarrow$ Fraction of successful episodes by model “.” over N episodes
for round $\leftarrow 1 \dots R$ **do**
 for each node m in M **do** ▷ bottom-up, left-right
 $M' \leftarrow$ Replace m in M with its left child
 if $\text{fitness}(M') \geq \text{fitness}(M)$ or $\text{SR}(M') \geq \text{SR}(M)$ **then**
 $M \leftarrow M'$
 else
 $M'' \leftarrow$ Replace m in M with its right child
 if $\text{fitness}(M'') \geq \text{fitness}(M)$ or $\text{SR}(M'') \geq \text{SR}(M)$ **then**
 $M \leftarrow M''$
 end if
 end if
 end for
end for

Essentially, Reward Pruning works by iteratively reducing the tree while monitoring its performance on the task. At each step, the algorithm selects an inner node and replaces it with its left child – if the performance of the tree is maintained or improved, the

Table 1
Parameters used in each environment for the MENS-DT-RL.

Parameter	Cartpole	Mountain Car	Lunar Lander	Maize Fertilization
ρ	0.6	0.8	0.6	0.6
F_b	0.98	0.98	0.5	0.98
F_d	0.05	0.1	0.05	0.05
P_d	0.2	0.4	0.2	0.2
k	1	3	5	1
K	1	1	1	1
α	0.1	0.1	2	0.1
Initial tree depth	3	3	3	3
Generations	200	200	200	50

replacement is kept, otherwise it is discarded and the same procedure is performed for the right child. If neither child proves to be an adequate replacement for the parent, the current step is terminated and the next node is selected. This process is repeated for all nodes in a bottom-up order until it reaches the root. At this point, every inner node has been evaluated at least once, so the procedure can either stop and return the resulting tree, or it can use this resulting tree as input for another “round” of pruning, starting from the bottom once again.

Reward Pruning employs two tests to decide whether a given pruning operation should be retained or reversed. Let M' be a smaller tree obtained by replacing one of the inner nodes of M with one of its children. The simplest way to decide whether to replace M with M' would be to check if $\text{fitness}(M') \geq \text{fitness}(M)$, since this would mean that performance has not decreased. However, RL tasks are often stochastic, which means that there is noise in the fitness calculation – even if M and M' were the same, their fitness could be different due to slight variations in episodes. For this reason, Reward Pruning considers not only the fitness but also the success rate of the model: that is, the fraction of episodes among the N executed that have reached the task’s predetermined minimum reward threshold. If M has either a lower fitness or success rate than M' , it is replaced. This provides a more reliable way to determine whether a particular pruning operation resulted in a better model.

Finally, note that the stochasticity of RL extends to both phases of Reward Pruning, making it a stochastic algorithm: given the same initial uninterpretable model, different runs of Reward Pruning may yield different intermediate trees and different final trees. This is consistent with common RL models and provides another key difference between Reward Pruning and traditional post-pruning algorithms.

4. Experiments and discussion

In this section, we want to answer the following questions:

1. How does MENS-DT-RL compare to other algorithms, in terms of produced tree size and performance?
2. What effect does the initialization have on the MENS-DT-RL’s results?
3. Can the resulting trees be interpreted?
4. Is there diversity in the solutions produced?

To this end, we conduct experiments with the MENS-DT-RL across four different RL environments: three benchmarks from the widely used OpenAI Gym framework [28], and an agronomical environment based on real-world crop management [29]. The results are compared with other methods from the literature, primarily through three different measures: episodic reward (which measures the performance of the agent in the task), tree size (which serves as a proxy for the interpretability of the resulting agent, with smaller trees being more interpretable), and success rate (which condenses the reward information by assessing the proportion of episodes that resulted in success). For every OpenAI Gym benchmark, a perfect success rate of 1.0 is attainable, as shown in the leaderboards available in the official documentation.² For the Maize Fertilization environment, to the best of our knowledge, the highest performance was achieved by the PPO agent released by the environment’s own authors, which reached a success rate of around 0.8.

Each algorithm was run for 50 simulations (30 in the case of crop management due to the simulator’s high computational cost), with each simulation resulting in a single univariate tree. To calculate the fitness of each individual during the execution of the algorithm, 100 episodes are run. All values reported were obtained by re-running the solutions on another 1,000 episodes. Furthermore, since the environments are stochastic, each tree has an average and a standard deviation for its reward and success rate, which means that the values reported here are the average of these averages and standard deviations. However, while the average behavior of each algorithm is useful for comparison, it is also important to understand that in practice only a single solution is needed, which is why we also report the best solution rather than the average. Table 1 contains the parameterizations for each environment, defined on the basis of preliminary tuning.

As described in Section 3, three different configurations for the MENS-DT-RL are considered, differing primarily in their initialization. The first configuration, called “MENS-DT-RL (R)”, is randomly initialized with randomly generated complete trees of

² <https://github.com/openai/gym/wiki/Leaderboard>.

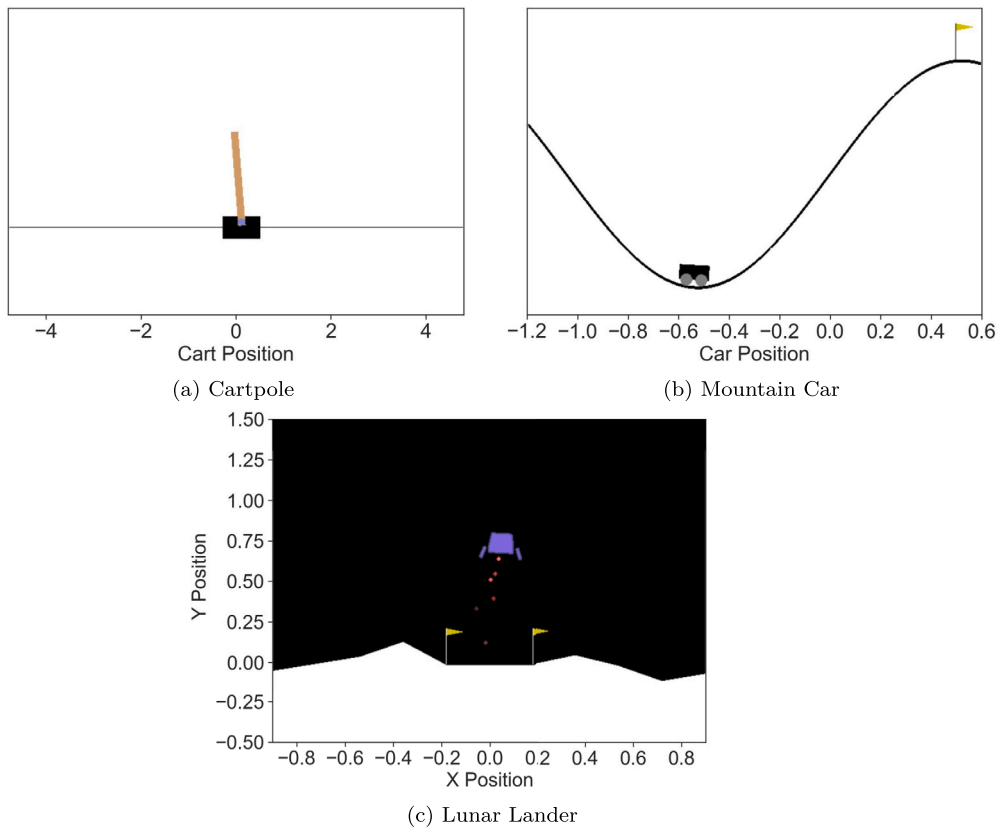


Fig. 4. Illustration of the three OpenAI Gym environments (Maize Fertilization is not pictured since it has no visual component).

depth 3 (i.e. 15 nodes) – this provides the algorithm with maximum diversity, but also no solution from which it can jumpstart its performance. The second configuration, referred to as “MENS-DT-RL (IL)”, initializes the population with trees obtained via IL. In particular, the i -th simulation of “MENS-DT-RL (IL)” is initialized with a population consisting of randomly generated trees of depth 3 and the i -th tree obtained from IL. The third and final evaluated configuration, called “MENS-DT-RL (P)”, follows the same scheme as the previous one but applies 10 rounds of Reward Pruning to the IL tree before adding it to the initial population. In addition, the experts used in IL are dense neural networks with 2 hidden layers of 32 nodes each, trained with deep Q-Learning and experience replay – the only exception is the crop environment, where we use the PPO model included in the library itself [29].

All three configurations are compared with different solutions from the literature. Given the novelty of interpretable AI and interpretable RL specifically, there are a scant few works which produce interpretable agents for the three environments employed here. The authors from [73], when faced a similar situation in their work on evolutionary trees for RL, remedied this by proposing an interpretability score that could be applied not only to DTs but to any machine learning model, therefore creating a common ground in which any pair of algorithms could be compared in terms of interpretability. Although this sounds intuitive, we avoid this approach for two main reasons. The first is that this does not provide any additional information of particular importance: in terms of interpretability, the trees indeed almost always surpass any of the neural network-based models that are popular in RL, but this is already known since DTs are widely known to be more interpretable than the alternatives. The second reason is that even in the rare case where the trees are indeed surpassed by some other method in terms of interpretability, for example, a medium-sized tree against a very small neural network, it is debatable whether the neural network truly could be considered “interpretable” in the common sense of the term, or if it is only achieving a better interpretability score simply because it is “gaming the metric” (e.g. it has fewer mathematical operations involved, while still having a hard-to-grasp multi-layered structure). As such, we opt to compare the DTs obtained here only with similar DTs from the literature.

All experiments were implemented on Python 3.10 programming language. The code is fully available at https://github.com/vgarciasc/CRO_DT_RL.

4.1. Environments

4.1.1. Cartpole

The Cartpole environment (illustrated on Fig. 4a) involves balancing a pole on a cart by moving the cart horizontally. The state consists of four different attributes: the cart’s position and velocity, and the pole’s angle and angular velocity. The actions involve either pushing the cart left or right. The reward is always +1 until the episode terminates, rewarding every step that the pole is kept

upright. The termination is achieved when the cart reaches the edge of the region of interest, if the episode reaches 500 steps, or if the pole's angle is greater than 12 degrees in any direction (in which case it is considered to have fallen). As such, the accumulated maximum reward for this environment is 500 points. In this work, we consider that an episode is successful if the accumulated reward is larger than 495 points.

4.1.2. Mountain Car

In the Mountain Car environment (illustrated in Fig. 4b) [81], the agent controls a car in a valley between two mountains, such that the goal is to make the car reach a flag on top of the rightmost mountain. Since the car does not have enough traction to climb this mountain by itself, it must carefully build up momentum to achieve a higher speed and reach the flag. Two attributes are available: the car's velocity, and its position along the X-axis. With regards to actions, there are three: accelerating to the left, to the right, or not accelerating at all. The reward is -1 for every step during which the flag is not reached, awarding agents that reach the flag faster. The episode terminates when the flag is reached or if 200 steps are executed, leading the worst possible solution to be an accumulated reward of -200 points. In this work, a successful episode of Mountain Car involves not only reaching the flag but doing so with an accumulated reward lesser than -110 points.

4.1.3. Lunar Lander

The Lunar Lander environment (illustrated in Fig. 4c) tasks the agent with landing a rocket upright on a flat surface. To this end, the agent must carefully balance each of its four actions: doing nothing, firing the main engine (located at the bottom of the ship), the left engine, or the right engine. Eight attributes are made available: the X and Y coordinates of the agent, its X and Y linear velocities, its angle and angular velocity, and two boolean attributes that signal whether or not each leg is touching the ground. Rewards for Lunar Lander involve several components. Crashing results in -100 points, while landing correctly results in +100 points. Each leg that contacts the ground results in +10 points. Furthermore, the agent must save fuel: it is penalized for -0.3 points each frame it fires the main engine, as well as -0.03 points for every side engine. The episode terminates if the agent crashes, if it lands correctly, or after 1000 timesteps. The episode is considered to be successful if the accumulated reward is larger than 200 points.

4.1.4. Maize Fertilization

The Maize Fertilization environment is a task modeled in gym-DSSAT [29], a Python RL wrapper for the widely celebrated Decision Support System for Agrotechnology Transfer (DSSAT; [82,83]) crop model. In this task, the goal is to design a nitrogen fertilization policy for a maize crop field, taking into account each day's data in order to determine how much nitrogen should be added to the soil. Adding no nitrogen may result in nitrogen deficiency and a reduced harvest, while adding excessive nitrogen involves a high financial cost and may result in water pollution, among other undesired effects. The agent must take all this into consideration to decide both (1) when to add fertilizer, and (2) how much fertilizer to add. Each episode corresponds to a growing season and contains around 160 days, with each day being a timestep. There are 12 different attributes each day, ranging from weather data (rainfall and temperature degrees) to current plant information (total biomass and growth stage). The reward is a function of both the plant nitrogen uptake and the quantity of fertilizer added, while the action at each step is a continuous value that denotes the quantity of fertilizer to add (from 0 to 200 kg/ha). For a more detailed description of the environment, we refer to [29]. In our paper, we define the success rate to be a final cumulative reward of 60.

4.2. Results

4.2.1. Cartpole

Cartpole is the simplest of the three environments, and it is expected that most approaches should perform well on it. Indeed, as it can be seen in Table 2, all three configurations of MENS-DT-RL were able to find a tree with 5 nodes and a perfect reward of 500.0 ± 0.0 , which means that for every episode out of 1000 tested, these best agents managed to balance the pole for the maximum amount of time (500 timesteps). Although all three configurations reached this solution, there are slight differences in the evolutionary process: Fig. 5(a) shows that the IL and pruning initializations start very close to the final solutions, while the randomly initialized MENS-DT-RL (R) needs some generations to catch up, since it starts from scratch. On average, the three configurations have already converged by generation 100, and simply maintain their results from then on. Furthermore, Fig. 5(c) shows that MENS-DT-RL does not get stuck on the initialized solutions, since both MENS-DT-RL (IL) and MENS-DT-RL (P) reduce their initial average tree sizes over time. In addition, this figure also shows that MENS-DT-RL (R) actually ends up with a smaller average tree size than the other two approaches, a fact reflected in Table 2 – this may be due to the small size of the best solution, which actually benefits from the random search aspect of this configuration. In terms of average reward and success rate, there is little difference between the averages of these three configurations, as they are all close to the perfect reward of 500.0 ± 0.0 . Given these results, MENS-DT-RL (R) appears to be the best configuration of the three, as it is able to reliably produce the best solution without incurring the cost of requiring an expert model.

Curiously, the existence of an optimal tree size does not imply the existence of a single optimal tree. By analyzing the results from the MENS-DT-RL configurations, we can see that there are different trees of size 5 that obtain a perfect success rate in the task: Fig. 6 shows three of them. The first one relies on Pole Angle and Pole Angular Velocity, however, the last two do not rely on the Pole Angular Velocity at all and instead use the Cart Velocity, differing between themselves with regards to how these two attributes are used. By analyzing the execution of these three solutions, it is indeed possible to see that there are differences in how they behave: as is shown in Fig. 7, trees A and B keep the pole in a tight range of $[-0.05, 0.05]$ radians, while tree C lets the pole reach larger angles

Table 2

Results for the Cartpole task. (*): as reported in the original paper. (†): re-executed on 1000 episodes based on the reported best solution from the original paper. (§): solution taken from [84]. Best solutions are highlighted in bold.

	Average results			
	Avg. Reward	Avg. Stdev. Reward	Average SR	Avg. #Nodes
Neural Network Expert	500.00	0.00	1.00	—
MENS-DT-RL (R)	499.92	0.33	1.00	6.40
MENS-DT-RL (IL)	499.98	0.27	1.00	6.84
MENS-DT-RL (P)	499.99	0.18	1.00	6.88
Custode and Iacca, 2023 [73]*	497.34	5.19	1.00	—

	Highlighted solutions			
	Avg. Reward	Std. Dev. Reward	Success Rate	#Nodes
MENS-DT-RL (R)	500.00	0.00	1.00	5
MENS-DT-RL (IL)	500.00	0.00	1.00	5
MENS-DT-RL (P)	500.00	0.00	1.00	5
Custode and Iacca, 2023 [73]*†	499.67	5.98	1.00	5
Silva et al., 2020 [20] †‡	499.80	2.71	1.00	13

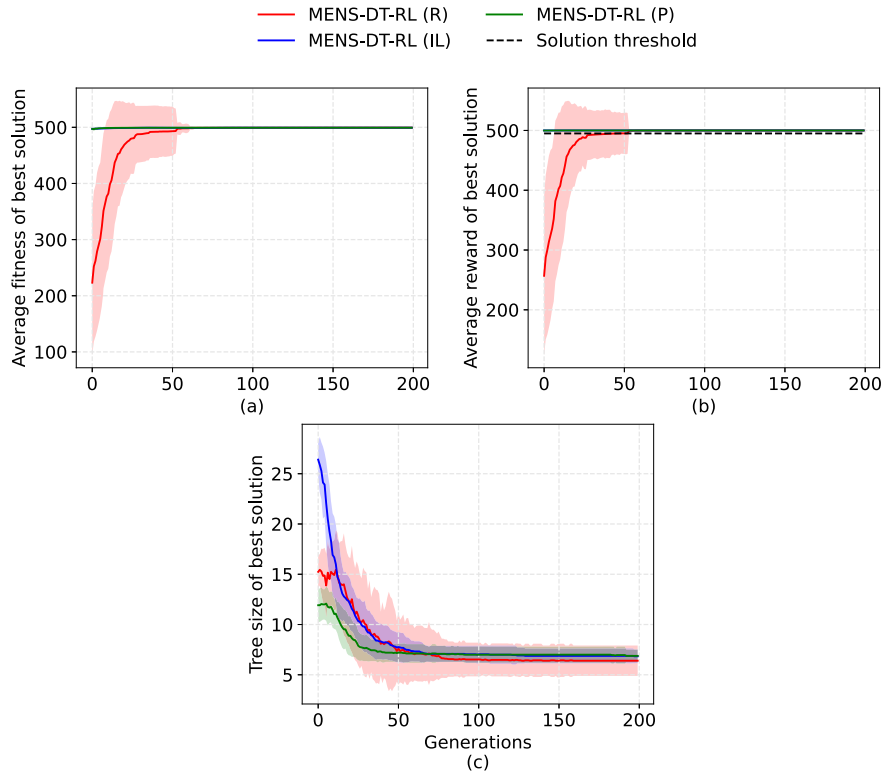


Fig. 5. Average metrics across generations for the fittest individual of each generation in Cartpole, across the three configurations of MENS-DT-RL. Each plot shows the evolution of a particular metric: (a) Fitness, (b) Reward, (c) Tree size. (For interpretation of the colors in the figure(s), the reader is referred to the web version of this article.)

before balancing it again. Furthermore, tree A has a slight preference for a right tilt, while tree B is the opposite. This exemplifies one of the key advantages of the evolutionary approach, which is the variety of produced solutions: due to the stochastic nature of evolutionary algorithms, the end-user has the power to choose an agent based on their own preferences of size, performance, and even features used.

To the best of our knowledge, two works report univariate DTs for the Cartpole task: [73] and [20]. The former does not report the average number of nodes of the produced trees but does report the average reward and standard deviation, which also closely approximate the values reported by the MENS-DT-RL (while being slightly lower). In addition, the authors report the best

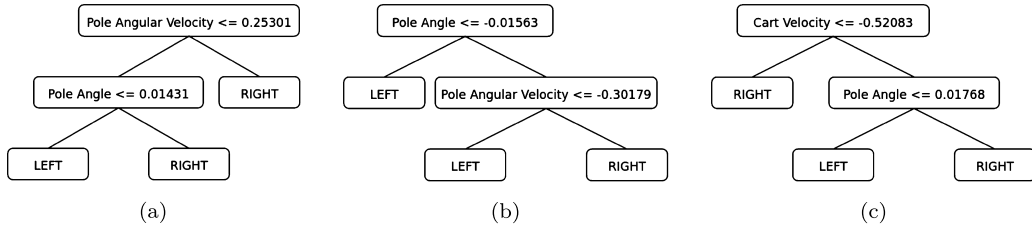


Fig. 6. Three of the best solutions obtained for Cartpole by the MENS-DT-RL approaches.

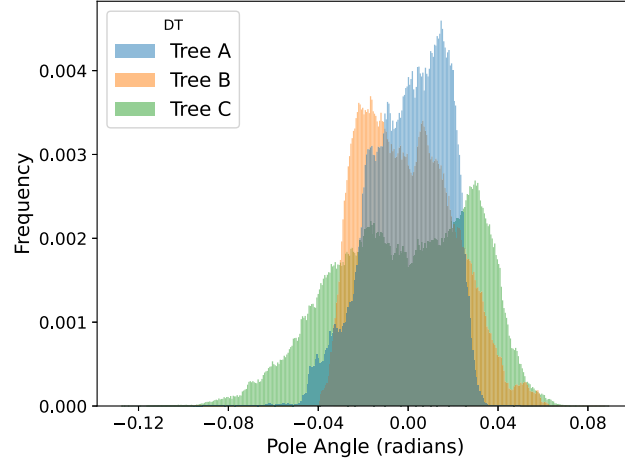


Fig. 7. Distribution of Pole Angle across 1,000 episodes by the three trees in Fig. 6.

solution found, which we re-implemented and executed for another 1000 test simulations to report here; this result not only achieves the maximum success rate but also contains 5 nodes, which reflects the best solutions found by MENS-DT-RL. Therefore, it seems highly likely that this is indeed the lowest size for which a univariate tree can achieve maximum reward for the Cartpole problem. Meanwhile, [20]’s gradient-based approach also obtained a solution with perfect reward, but with 13 nodes – higher than the solutions found by MENS-DT-RL and [73].

4.2.2. Mountain Car

Mountain Car is a simpler problem than Cartpole in terms of state attributes, but due to its sparse rewards it is actually harder to solve: a good solution that almost touches the flag and a bad solution that never leaves the starting position both have the minimum reward of -200. Fig. 8 compares the average performance of the three configurations over the evolutionary process. As it can be seen, MENS-DT-RL (R) is able to greatly improve its fitness by increasing the average reward and reducing both the tree size and the standard deviation of the reward, but although its final trees are only slightly larger than those of the other two configurations, its average reward pales in comparison and fails to reach the task solution threshold.

On the other hand, both MENS-DT-RL (IL) and MENS-DT-RL (P) are able to produce good quality results. Table 3 shows that the two initializations produce solutions with an average success rate of 0.74, a value that greatly exceeds the success rate of 0.28 obtained by the MENS-DT-RL (R). Moreover, note that this result is obtained even though the DNN expert has a success rate of only 0.39, indicating that the initial solutions do not need to be perfect to benefit from MENS-DT-RL. This greatly reduces the effort required to train the expert, which in turn reduces the computational cost of using IL-based initialization and makes it easier to use. Fig. 8 further shows that MENS-DT-RL improves these two initializations, as the average fitness increases even though it starts in a good range. This improvement is mainly due to a reduction of the average standard deviation, which means that the solutions become more consistent in their performance, even though the average reward is maintained. Note that if the fitness did not include the standard deviation, MENS-DT-RL would ignore it and such an improvement in consistency would not take place.

Table 3 shows the best solutions obtained in this environment. As it can be seen, all three configurations are able to produce a tree with 9 nodes, an approximate reward of -102 ± 0.7 , and a success rate of 1.0 – even the randomly initialized MENS-DT-RL (R), which has a worse performance on average. Note that this solution is not only small enough to be highly interpretable, but also has a better success rate than the neural network expert: given the previous remark that both MENS-DT-RL (IL) and MENS-DT-RL (P) have a higher average performance than MENS-DT-RL (R), it can be concluded that the evolutionary algorithm effectively takes the decent solutions from the expert and improves them towards an optimal behavior. In addition, these algorithms also find some solutions that although not the best in terms of fitness, are also very noteworthy: trees that also have a perfect success rate of 1.0 but with a smaller size of 7 nodes (see Table 3). Their average reward of about -106 ± 1.3 is slightly worse than that of the fittest solutions, but depending on the user’s priorities, they might actually be preferable due to their smaller size and similar success rate.

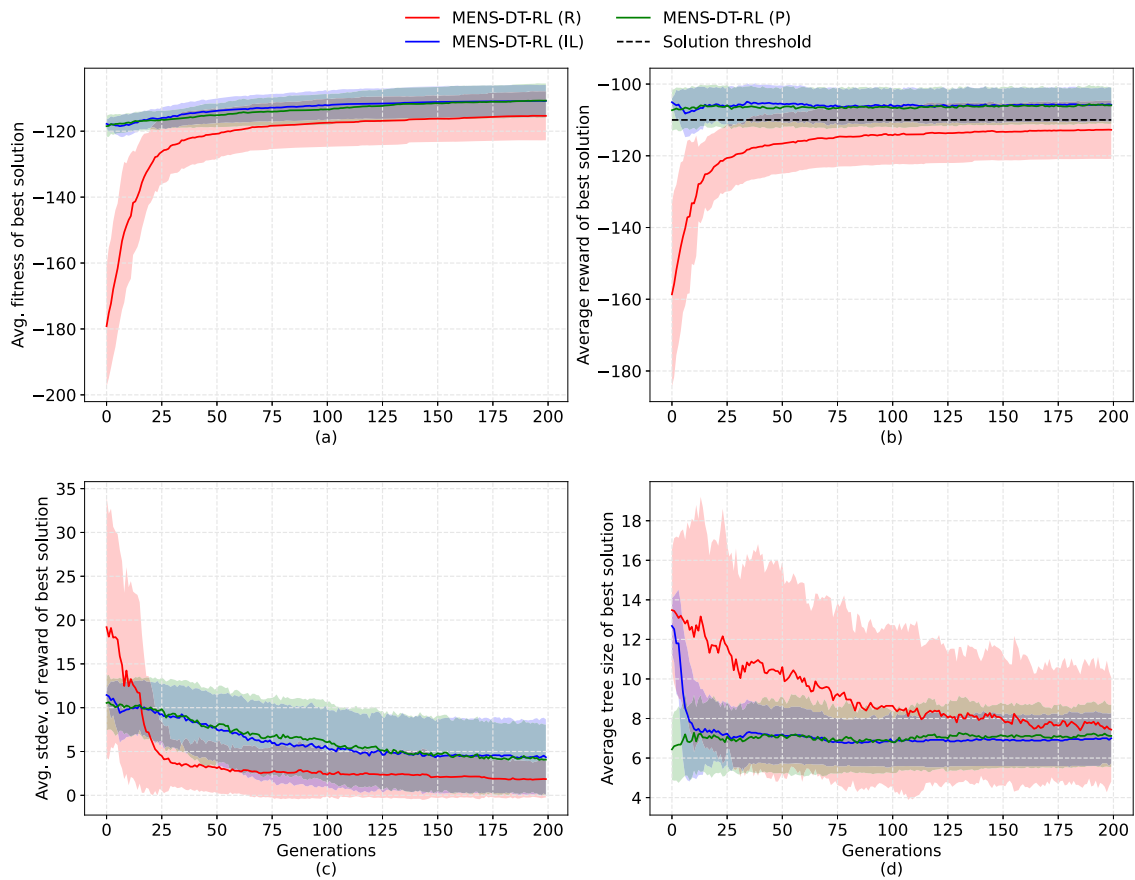


Fig. 8. Average metrics across generations for the fittest individual of each generation in Mountain Car, across the three configurations of the MENS-DT-RL. Each plot shows the evolution of a particular metric: (a) Fitness, (b) Reward, (c) Standard Deviation of Reward, (d) Tree size.

Table 3

Results for the Mountain Car task. (*): as reported in the original paper. (**): re-executed on 1000 episodes based on the reported best solution in the original paper. Best solutions are highlighted in bold.

	Average results			
	Avg. Reward	Avg. Stdev. Reward	Average SR	Avg. #Nodes
Neural Network Expert	-113.03	24.08	0.39	–
MENS-DT-RL (R)	-112.98	2.08	0.28	7.44
MENS-DT-RL (IL)	-106.30	4.49	0.74	7.00
MENS-DT-RL (P)	-106.68	5.94	0.74	7.12
Custode and Iacca, 2023 [73] *	-108.16	–	0.70	–
	Highlighted solutions			
	Avg. Reward	Std. Dev. Reward	Success Rate	#Nodes
MENS-DT-RL (R)	-102.53	0.90	1.00	9
MENS-DT-RL (IL)	-102.79	0.71	1.00	9
MENS-DT-RL (P)	-102.79	0.70	1.00	9
MENS-DT-RL (IL)	-106.54	1.22	1.00	7
MENS-DT-RL (P)	-106.46	1.33	1.00	7
Custode and Iacca, 2023 [73] **	-101.72	3.04	1.00	13
Dhebar et al. 2020 [74] *	-105.82	21.77	0.83	–

This is another example of the diversity of solutions obtained by the MENS-DT-RL approach, and of the choices that this gives to the end user. These solutions are shown in Fig. 9.

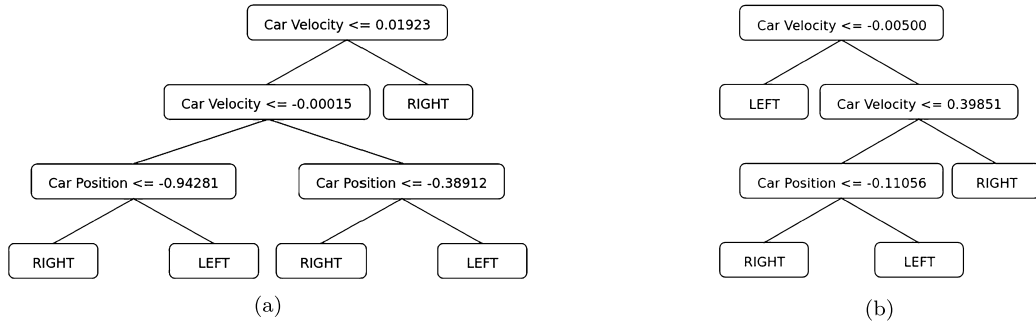


Fig. 9. Two of the best solutions obtained for Mountain Car by the MENS-DT-RL approaches (obtained by MENS-DT-RL (R) and MENS-DT-RL (P), respectively).

Finally, we compared our results with those reported in the literature. Our main point of comparison is the work of [73], which, to the best of our knowledge, is the only contribution that has experimented with univariate trees for the Mountain Car environment. The authors reported both a lower average reward and a lower success rate than those obtained by our two most sophisticated initializations, which indicates the advantage of using MENS-DT-RL, but it must be noted that in their work only 10 simulations were performed, while our results are the average of 50. Given this situation, it might be more informative to look at the best solution. The best solution found by [73] was a tree with 13 nodes and a perfect success rate of 1.00, which can be directly compared with the trees obtained in Fig. 9, both of which have a perfect success rate while having a smaller tree size (4 and 6 fewer nodes, respectively). This indicates that MENS-DT-RL is able to produce more interpretable solutions without compromising performance, even when randomly initialized. Moreover, we note that in [74] the authors also experimented with DTs for the Mountain Car environment, but instead of using univariate splits, they used highly nonlinear splits, which simply do not have the same level of interpretability as traditional univariate DTs. To the best of our knowledge, the trees reported here with maximum success rate and 7 or 9 nodes are the best interpretable DT solutions for the Mountain Car benchmark.

4.2.3. Lunar Lander

Lunar Lander is the most complicated environment of the benchmarks presented here, with 8 attributes, 4 different actions, and a movement dynamic that is much more complex than that of the previous two tasks. Such complexity may not be a problem for a model like the expert DNN (which achieves a success rate of 0.97), but it poses a clear challenge to the simplicity of univariate DTs, especially in terms of tree size: good solutions require much larger trees than those developed for Cartpole and Mountain Car.

Table 4 shows the results for this environment, where it can be seen that MENS-DT-RL (P) outperforms both the random and the IL initializations. While the randomly initialized MENS-DT-RL (R) is able to produce good solutions for Cartpole and Mountain Car, in Lunar Lander it is unable to evolve any tree with a success rate higher than 0.1 and ends up with an average success rate of 0. Such a negative result is a direct consequence of the complexity of the environment: since good solutions have more nodes and are easily broken by even small changes, it is strikingly difficult for the evolutionary approach to find such solutions when starting from scratch; that is, the topology of the solution space is not amenable to encountering good solutions based on what amounts to random search. Fig. 11(b) shows that while the average reward of this configuration improves over time, it is far from the task solution threshold of 200.

Table 4

Results for the Lunar Lander task. Best solutions in terms of fitness are highlighted in bold.

	Average results			
	Avg. Reward	Avg. Stdev. Reward	Average SR	Avg. #Nodes
Neural Network Expert	254.32	31.71	0.97	—
MENS-DT-RL (R)	-52.39	32.39	0.00	5.76
MENS-DT-RL (IL)	242.13	55.68	0.92	278.76
MENS-DT-RL (P)	223.46	77.10	0.84	50.04
	Highlighted solutions			
	Avg. Reward	Std. Dev. Reward	Success Rate	#Nodes
MENS-DT-RL (R)	-54.93	128.12	0.06	7
MENS-DT-RL (IL)	245.12	40.50	0.93	187
MENS-DT-RL (P)	237.70	72.51	0.91	37
Silva et al. 2020 [20]	-78.40	32.20	0.00	19
Dhebar et al. 2020 [74]	234.98	22.25	0.99	—

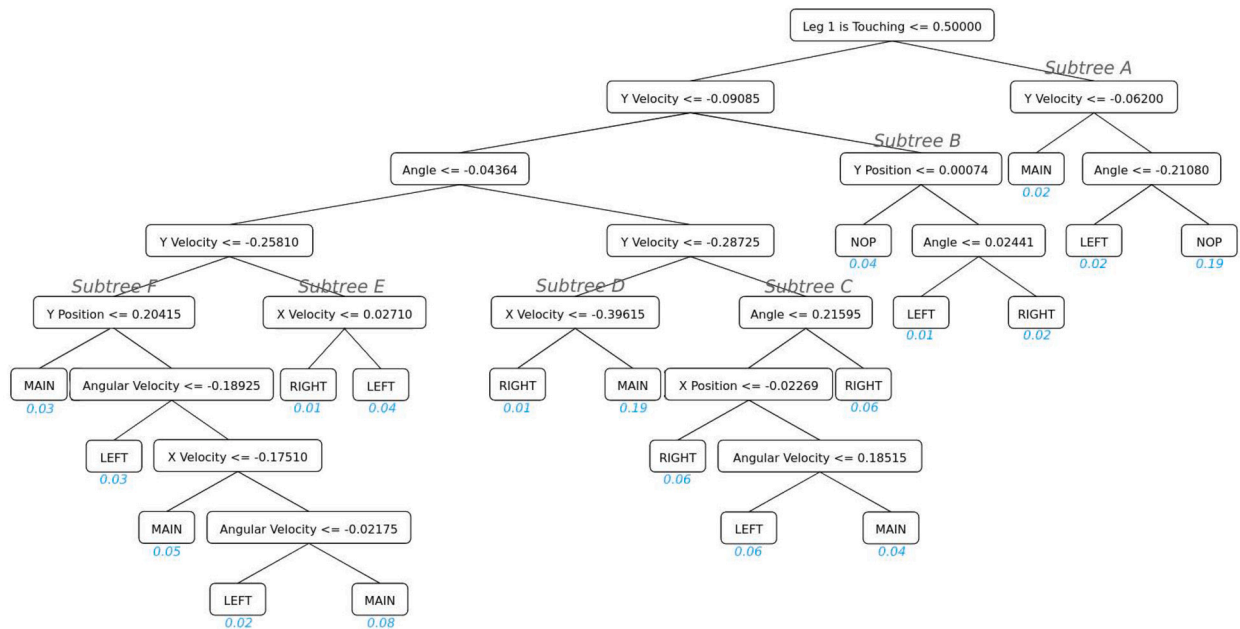


Fig. 10. Best solution found for the Lunar Lander environment, obtained by MENS-DT-RL (P). Labels in blue are the percentage of visits for every leaf.

Initializing MENS-DT-RL with IL solutions, however, yields more interesting results: the average success rate is 0.92 with an average tree size of 278.76, which means that MENS-DT-RL reduces the tree size of the IL trees while sacrificing some small amount of performance. This is illustrated in Fig. 11, where it can be seen that the average reward of MENS-DT-RL (IL) is mostly stagnant over time, while its size continues to be reduced. This shows that MENS-DT-RL is indeed capable of handling solutions to a more complex task like Lunar Lander, but it needs good initial solutions from which to start its search. However, the trees generated by this configuration are too large to be interpreted, which explains why Fig. 11(a) shows that MENS-DT-RL (IL) has a much lower average fitness than the poorly performing MENS-DT-RL (R).

This challenge can be overcome by using a pruned initialization. Since Reward Pruning reduces the size of the IL trees while trying to maintain their performance, MENS-DT-RL (P) is given a much better fitness from the start, leading it to outperform the other two configurations. Furthermore, Fig. 11 shows that this fitness continues to improve over time, mainly due to further reductions in the tree size and the standard deviation of the reward, i.e. the solutions become smaller and more consistent. While the average success rate of MENS-DT-RL (P) is slightly lower than that of MENS-DT-RL (IL) (0.84 vs. 0.92), its average tree size is an order of magnitude smaller (50.04 vs. 278.76), resulting in trees that not only frequently solve the task, but are also in a better range of interpretability. Furthermore, it is also important to look at the best solution, since this is what the end user will actually use: this solution, shown in Fig. 10, has a 91% success rate and a tree size of 37 nodes, a combination of interpretability and performance that, to the best of our knowledge, has not yet been achieved by any other work.

Three papers in the literature have tackled the Lunar Lander environment with DTs. Silva et al. [20] applied univariate trees to the task in the form of rule lists, but were unable to obtain solutions that solved the problem, reporting an average reward of -78.4 with a standard deviation of 32.2. Similarly, Custode and Iacca [73] attempted to extend their evolutionary approach to this task, but did not find a configuration that gave satisfactory results, reporting only a multivariate tree instead. Finally, Dhebar et al. [74] reported a DT with a success rate of 0.99 that consistently solves the problem, but it employs several nonlinearities that call into question the interpretability aspect. To the best of our knowledge, this is the first time that a high-performing interpretable solution using univariate DTs has been proposed for this benchmark.

4.2.4. Maize Fertilization

The last of the four simulated environments is Maize Fertilization, a task made difficult not only by its large state space (12 different attributes) and complex internal dynamics (controlled by the DSSAT crop management system) but also by the continuity of its actions: at each step of the simulation, instead of choosing from a list of predetermined actions, the agent must specify a quantity (how much fertilizer to place in the soil that day). Although this is quite natural for a complex neural network to represent, it poses a challenge for DTs: while it is possible to modify the leaves of the tree to represent continuous values instead of categorical ones (as in Regression Trees), this limits the possible outputs of the tree to the number of different leaves it has, and thus puts smaller trees at a disadvantage with regards to action sensitivity. Some authors try to bypass this restriction by including linear models in the leaves (the so-called Model Trees [16]), however since this approach compromises interpretability, in this paper we chose to restrict every leaf to a single value and grow the tree if needed.

Table 5 contains the simulation results. As it can be seen, the neural network expert is able to achieve a 0.80 success rate, with an average reward of 59.22 which is very close to the success threshold of 60. This is expected of deep RL models and indeed closely

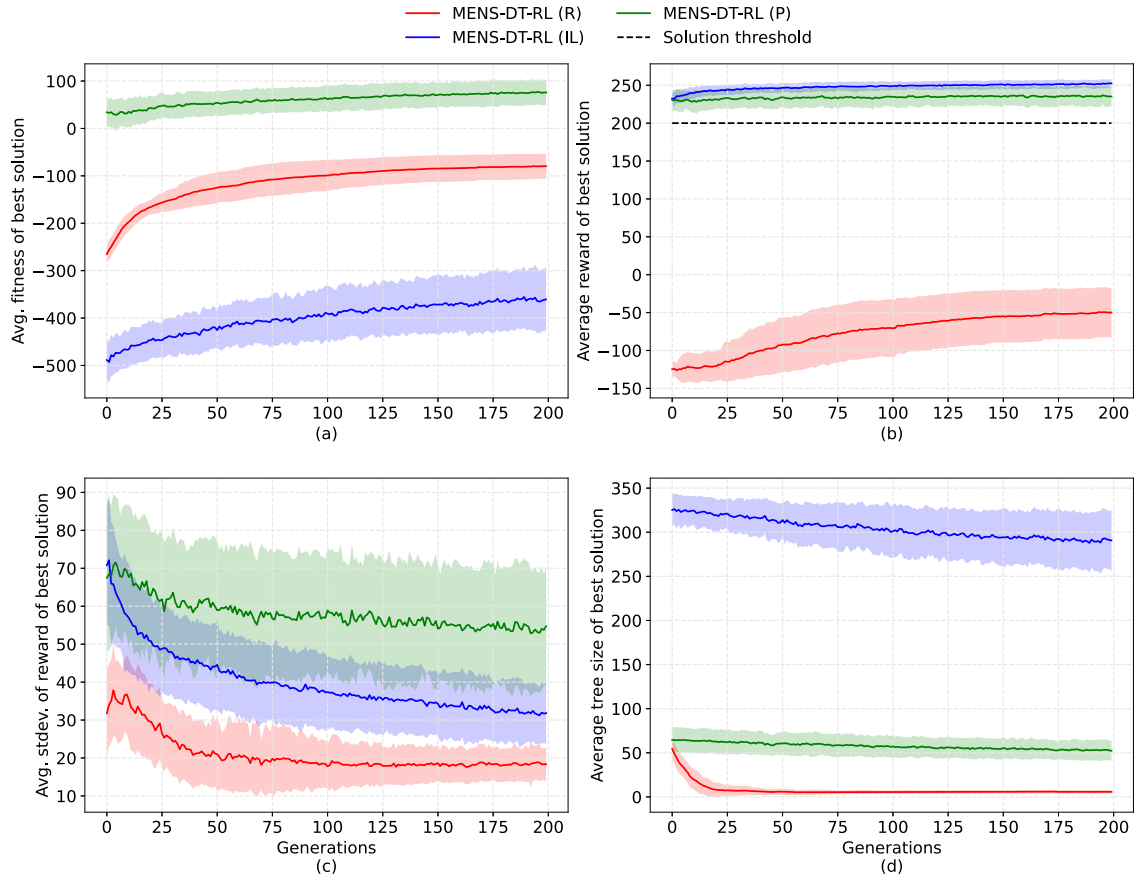


Fig. 11. Average metrics across generations for the fittest individual of each generation in Lunar Lander, across the three configurations of MENS-DT-RL. Each plot shows the evolution of a particular metric: (a) Fitness, (b) Reward, (c) Standard Deviation of Reward, (d) Tree size.

mirrors the benchmark results showed in [29]. In contrast, MENS-DT-RL is unable to achieve such a high-performing behavior by itself: its random initialization version has an average reward of 2.64 and an average success rate of 0.35, both of which fall far below the solution quality established by the expert. As it has been shown in previous environments, leveraging this expert is key to achieving much better solutions. By using Imitation Learning, MENS-DT-RL (IL) is able to obtain a similar performance as the expert while bringing the policy to the DT domain, attaining a success rate of 0.80 and an average reward of 57.33. However, it is still questionable if these solutions can be clearly interpreted since the average tree size is 69.73. MENS-DT-RL (P) is able to bring a final improvement through the Reward Pruning approach, maintaining a near-identical success rate as MENS-DT-RL (IL) and a very similar average reward (56.74 against 57.33), all the while achieving an average tree size of 9.93 that is much more within the realm of ordinary DT interpretability. Fig. 12 reinforces these conclusions: MENS-DT-RL (P) has a very similar reward to MENS-DT-RL (IL), all the while having a similar tree size to MENS-DT-RL (R). Furthermore, the fitnesses of all three configurations have different rates of increase over time: MENS-DT-RL (R) is the fastest-growing of the three, since it has so much to improve, and conversely MENS-DT-RL (P) is the slowest, since the Reward Pruning solutions it starts from are already of very high quality. MENS-DT-RL (IL) falls squarely in the middle, since its initial solutions have good reward but high tree size.

The same relationship between models can be found by looking at the best solutions obtained by each configuration. Although MENS-DT-RL (R) has a low average performance, its best solution is quite decent: a tree of 5 nodes and success rate of 0.70. Considering the results from previous environments, we get a further indication that MENS-DT-RL (R) is able to eventually find good-quality solutions given that their tree size is not too high (as happens for Cartpole and Mountain Car, but not for Lunar Lander). As for MENS-DT-RL (IL), its best solution has a slightly higher success rate as the expert, with a tree size of 13 nodes. The best solution overall comes from MENS-DT-RL (P), which maintains this high success rate while having a reduced tree size of only 5 nodes. This tree is depicted in Fig. 13.

We compare this DT agent against the two baseline policies proposed in the original paper: Null Policy and Expert Policy [29]. *Null Policy*, as the name suggests, consists of adding no fertilizer throughout the period, therefore incurring no fertilizer costs at the expense of a worse harvest – this can be clearly seen in Fig. 14, where the policy’s cumulative reward is shown to be entirely non-decreasing. *Expert Policy*, on the other hand, adds a fixed amount of fertilizer on three pre-specified days of the growing season (based on the experiment done by [85]): this results in the three drops in Fig. 14, which decrease reward in the short term but end up resulting in a final larger cumulative reward than the Null Policy. These two policies can be easily represented by DTs of size 1

Table 5

Results for the Maize Fertilization task. Best solutions in terms of fitness are highlighted in bold. (*): Number of tree nodes of the policies if they were represented as DTs.

	Average results			
	Avg. Reward	Avg. Stdev. Reward	Average SR	Avg. #Nodes
Neural Network Expert	59.22	13.77	0.80	—
MENS-DT-RL (R)	2.64	27.71	0.35	12.13
MENS-DT-RL (IL)	57.33	12.83	0.80	69.73
MENS-DT-RL (P)	56.74	12.19	0.79	9.93

	Highlighted solutions			
	Avg. Reward	Std. Dev. Reward	Success Rate	#Nodes
MENS-DT-RL (R)	56.16	18.15	0.70	5
MENS-DT-RL (IL)	57.38	11.74	0.83	13
MENS-DT-RL (P)	57.70	11.23	0.83	5
Null Policy [29]	42.49	5.95	0.08	1*
Expert Policy [29,85]	55.22	29.44	0.59	7*

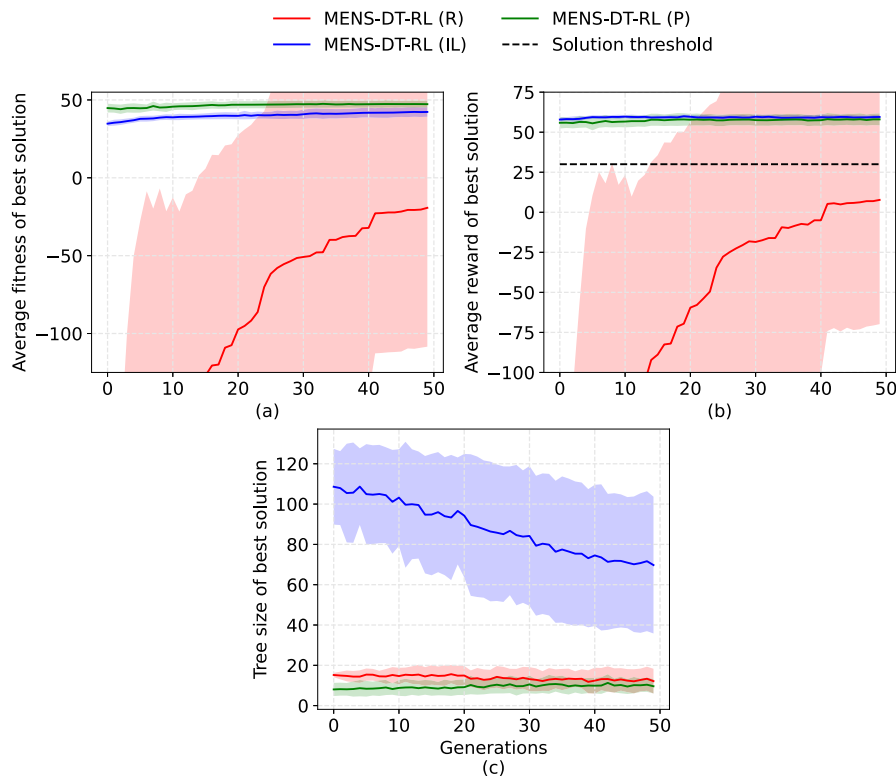


Fig. 12. Average metrics across generations for the fittest individual of each generation in the Maize Fertilization environment, across the three configurations of MENS-DT-RL. Each plot shows the evolution of a particular metric: (a) Fitness, (b) Reward, (c) Tree size.

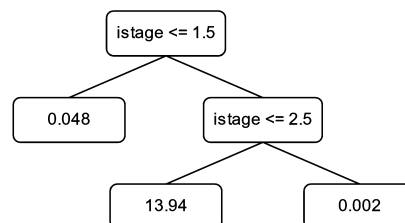


Fig. 13. Best solution found for the Maize Fertilization environment (obtained by the MENS-DT-RL (P)).

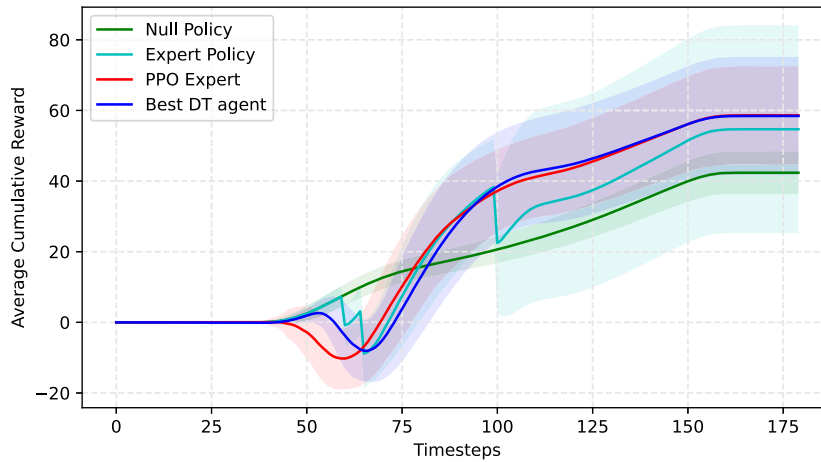


Fig. 14. Average cumulative reward over time of four different policies on the Maize Fertilization environment. This figure is based on a similar analysis done in the original gym-DSSAT paper [29]. Metrics were measured over 1,000 episodes.

and 7, respectively. Also shown in the figure is the *PPO Expert*, which refers to the model we have used so far as an expert; instead of adding fixed amounts on fixed days, it takes a subtler approach by adding gradual quantities over a large number of days, resulting in a smoother curve and a higher cumulative reward over time. Finally, the *Best DT Agent* has a curve whose smoothness closely mirrors the PPO's, although it is clear that this agent begins fertilization sooner. Despite this difference, the two end up with very similar final cumulative rewards, with the key distinction that the DT agent is interpretable, while the PPO is not.

4.3. Interpreting the models

With the goal of demonstrating the interpretability of the univariate tree approach to RL, this section provides interpretations for the best solutions obtained in each environment. These interpretations have been supported by a visualization tool (included in the previously-mentioned public repository) that displays the DT agent and the rendered environment side-by-side, allowing the user to see in real-time which nodes are activated at each time step (see Fig. 15). Editing the tree is also possible, paving the way for interactions typical of human-in-the-loop machine learning [86]. We argue that tools like this highlight the interpretability of DT models and their usefulness in the RL domain, allowing user insights that are impossible in uninterpretable models such as deep neural networks.

4.3.1. Cartpole

For the Cartpole task, we have chosen Tree B in Fig. 6 to interpret. Since the model is small, the interpretation is quite simple: if the angle of the pole is less than or equal to -0.015 , the pole is tilted to the left, so the cart must move to the left to correct the pole. On the other hand, if the pole's angle is greater than -0.015 , then it is either perfectly balanced or tilted to the right. In this case, the tree uses the angular velocity to decide what to do: if the pole is increasingly tilted to the left (Angular Velocity ≤ -0.30), then the cart must move to the left to correct it since just trusting the angle and moving to the right would increase the velocity even more and result in a difficult-to-recover scenario. Otherwise, the cart must move to the right. This behavior results in a left tilt bias, which can be seen in Fig. 7.

4.3.2. Mountain Car

For the Mountain Car task, we have chosen to interpret Tree A in Fig. 9. The first check made by the agent is to see if the car is moving to the right (Car Velocity ≤ 0.019): if it is, then it maintains this momentum by moving even further to the right, which means it either reaches the final flag or reaches a slope where it loses momentum and starts moving in the opposite direction. If this test fails, then the movement must be defined by whether the car is stationary or moving to the left, two situations distinguished by the following "Car Velocity ≤ -0.0 " split. The car then moves to the left in two situations: when it is already moving to the left but has not yet reached position -0.94 , or when it is stationary and its position is greater than -0.38 . Fig. 4b shows where these two positions are. They both correspond to points on the slopes just *before* the car's momentum drops to zero, so by explicitly reversing direction in these two situations, the agent is able to start moving in the opposite direction before the slope forces it to do so, thus optimizing the time needed to build up enough momentum to reach the flag.

4.3.3. Lunar Lander

The Lunar Lander agent is the most difficult of the four to interpret, mainly due to its larger size. To do this, it is crucial to identify which subtrees are responsible for handling which situations, which can be done by visualizing the agent in action and observing the leaf activation patterns. Fig. 10 encapsulates this idea by both highlighting specific subtrees and annotating each leaf with its fraction of visits, indicating which are more frequently activated.

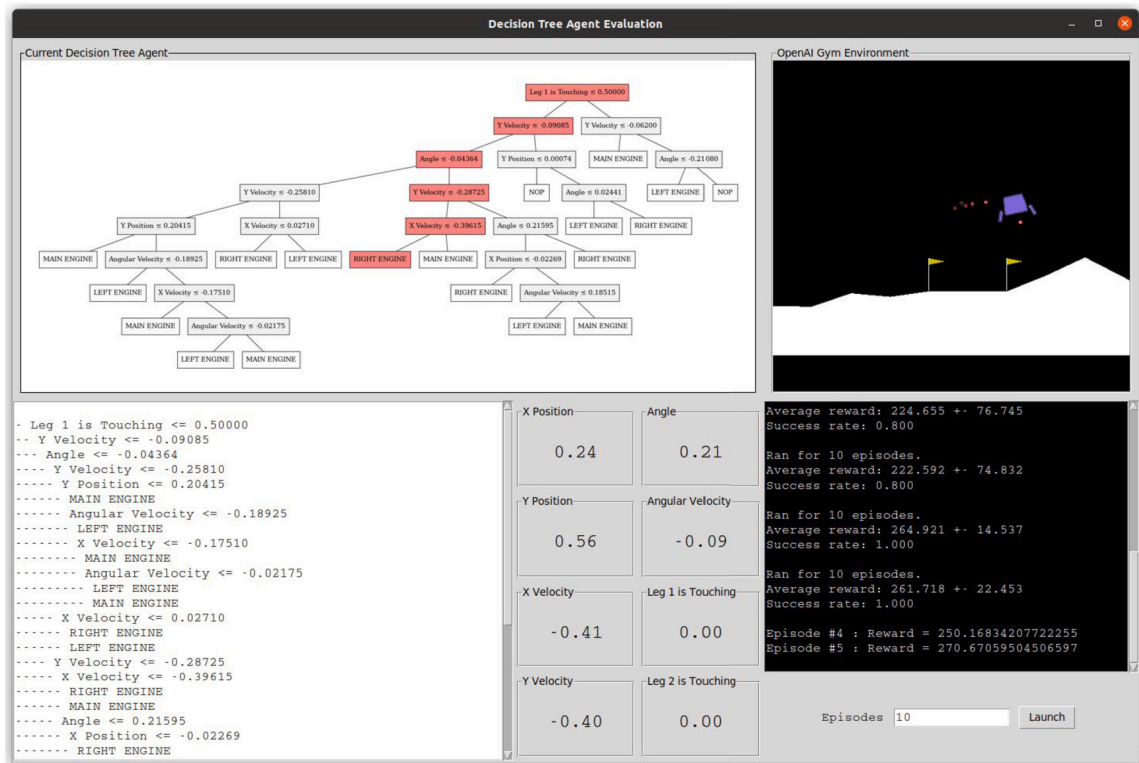


Fig. 15. Visualization tool that facilitates investigating the agent’s reasoning and interacting with its rules.

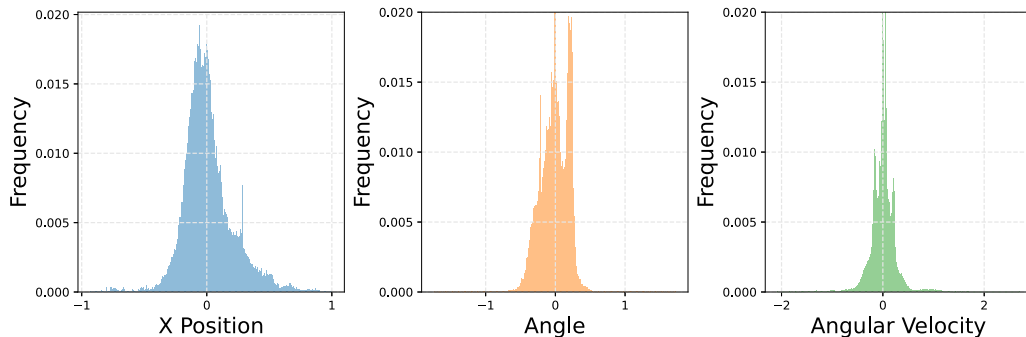


Fig. 16. Distribution of attributes from the states visited by the Lunar Lander agent in Fig. 10. Note that there is an asymmetry in the angles, as well as a more prevalent presence in the positive values of X than in the negative ones.

The root split in the tree checks whether Leg 1 touches the ground, leading to Subtree A if the result is positive. It then becomes clear that this subtree is responsible for the final landing procedures, activating the main engine if the agent is falling too fast, correcting with the left engine to land the other leg if the ship is tilted too much to the right, and otherwise doing nothing once the landing is complete. Because the task does not end immediately in this latter case, the agent spends some time doing nothing on the landing pad, resulting in the high frequency of the NOP leaf in this subtree.

Subtree B, on the other hand, is activated when the agent is not touching the ground but has a vertical velocity greater than -0.09: that is, it is falling slowly. As such, it handles a pre-landing phase in which the agent has started to decelerate but is not yet close enough to the ground for Subtree A to handle the situation. Quite simply, Subtree B either fires the left or right engines symmetrically based on the current angle of the ship, or it does nothing if the vertical position is less than 0.0 (i.e. the ship is extremely close to touching the ground, but has not yet done so).

Then, the remaining subtrees are divided into two groups based on the split Angle ≤ -0.04 : the left group handles situations where the angle is mostly negative (i.e., right-tilt), and the right group handles situations where the angle is mostly positive (i.e., left-tilt). The latter responds for 42% of the actions taken by the agent, while the former responds for only 26%, suggesting a slight angle asymmetry during flight. This suspicion is further supported by Fig. 16, which shows the distributions of the states visited by

the agent – there is a slight tendency for positive angles, as well as a better distributed presence in the rightmost half of the scene. In this context, it is easier to interpret Subtrees C and D. Subtree D is activated when the vertical velocity is less than -0.28 (i.e. the agent is falling too fast). In this situation, the most sensible thing to do is to decelerate by firing the main engine as soon as possible – and indeed, the corresponding main engine leaf in Subtree D is one of the two most frequent nodes in the tree, serving as the main way to mitigate the fall. However, if there is no danger of free fall (Y Velocity > -0.28), then Subtree C corrects the angle and position: it fires the right engine to correct either a highly positive angle (Angle > 0.21) or an off-center position (X Position ≤ -0.02). Otherwise, this subtree corrects the natural tendency to tilt left by firing the left engine if the angular velocity allows it (Angular Velocity ≤ 0.185).

The last pair of subtrees is also responsible for deceleration, angle correction, and position correction, but in a slightly different way to account for the fact that it only handles negative angles. If the vertical speed is high enough (i.e., the agent is not falling too fast), Subtree E corrects the horizontal speed in a remarkably symmetrical way, firing the engine in the opposite direction to the current one. However, if the agent is in free fall, Subtree F is activated to slow down, just like Subtree D – although it performs this role in a slightly more sophisticated way. The subtree not only fires the main engine to reduce the rate of fall, but it can also fire the left engine in two situations: first, if the angular velocity is extremely skewed to the left (Angular Velocity ≤ -0.18), or if it is slightly skewed to the left and there is no danger of going too far away from the landing pad (Angular Velocity ≤ -0.02 and X Velocity > -0.17). In all other situations, the main engine is actually fired for deceleration. Empirically, firing the left engine in Subtree F usually results in the angle being tilted to the other side, thus ceding control to Subtrees C and D.

4.3.4. Maize Fertilization

Finally, we interpret the best DT produced for the Maize Fertilization environment (see Fig. 13). The model has three possible actions: add a very small amount of nitrogen to the soil (0.048), add a larger amount of nitrogen (13.94), or add close to no nitrogen (0.002). In deciding which of these amounts to add, the model only considers the *istage* attribute, which corresponds to the growing stage of the corn; more specifically, the model adds a very small amount of nitrogen when the plants are at the end of their juvenile stage (i.e., when *istage* = 1), a larger amount of nitrogen when 50% of them have completed their flowering initiation (i.e., when *istage* = 2), and almost no fertilizer in all other cases. This strategy bears some similarities to the expert policy described in Section 4.2.4 (which also adds fertilizer only during the first half of the crop's development), but both differ significantly in terms of nitrogen distribution: rather than following the expert policy's approach of adding large amounts of fertilizer over a few days, the DT policy opts to add smaller amounts of fertilizer interspersed throughout all of the crop's early growth stages. Significantly, no information other than the current growth stage, not even weather or soil data, is required to achieve this level of performance; this provides insight not only into the problem but also into the inner workings of the simulator itself. We emphasize that this kind of insight comes naturally when dealing with interpretable models such as DTs, while it is much more difficult or even impossible for uninterpretable models such as deep neural networks.

5. Conclusion

In this work, we proposed a novel algorithm called MENS-DT-RL to produce interpretable DTs for RL tasks. It works by extending the CRO-SL optimization ensemble algorithm with novel operators capable of handling DT structures, as well as with a fitness metric that prioritizes interpretable and high-performing models. We also proposed three different initializations for MENS-DT-RL, including the usage of IL techniques to provide good initial solutions, and a novel pruning approach called Reward Pruning that reduces RL trees while maintaining their performance.

The algorithm and its configurations have been evaluated on four environments: three benchmarks from the widely used OpenAI Gym taskset, and a crop fertilization environment, based on a real-world problem. The results showed that the proposed MENS-DT-RL algorithm was able to produce high-quality interpretable models for all environments, to the best of our knowledge outperforming the state-of-the-art of DTs for RL on three of them (Mountain Car, Lunar Lander and Maize Fertilization). In particular, the MENS-DT-RL with random initialization was able to achieve the best solution for Cartpole and Mountain Car (1.0 success rate with 5 and 7 nodes, respectively), while the proposed Reward Pruning approach was required to achieve the best solutions for Lunar Lander (0.9 success rate with 37 nodes) and Maize Fertilization (0.8 success rate with 5 nodes). The best models for each environment were discussed in detail and it was shown that they are indeed interpretable, indicating that the proposed approach is capable of achieving interpretability without losing performance. In addition, it was shown that the best solutions obtained by the algorithm sometimes differ in their approach to the problem, effectively empowering the user with the choice of which model to choose. Overall, the results show that MENS-DT-RL excels at tasks both artificial and real-world-based, handling simple and complex problems, either with discrete or continuous action spaces.

As future work, it would be interesting to extend the MENS-DT-RL to other types of RL problems, such as those with visual attributes as input (instead of tabular data), and with multiple agents (instead of a single one). In addition, it would be worthwhile to study Reward Pruning in more detail and to experiment with other designs, since this approach was crucial in solving the more complex Lunar Lander task. Finally, it might be interesting to extend the MENS-DT-RL to use other DT models, such as allowing splits that contain small and interpretable combinations of attributes (such as logical operators between boolean attributes).

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests:

Vinícius G. Costa reports financial support was provided by Coordination of Higher Education Personnel Improvement. Carlos E. Pedreira reports financial support was provided by National Council for Scientific and Technological Development. Carlos E. Pedreira reports financial support was provided by Carlos Chagas Filho Foundation for Research Support in the State of Rio de Janeiro. Sancho Salcedo-Sanz reports financial support was provided by Spain Ministry of Science and Innovation. Jorge Pérez-Aracil reports financial support was provided by Spain Ministry of Science and Innovation.

Data availability

The code is in a public repository shared in the paper.

Acknowledgements

This work was supported in part by the Brazilian research agencies CNPq — National Council for Scientific and Technological Development (Grant Number 306258/2019-6); FAPERJ—Foundation for Research Support of Rio de Janeiro State (Grant Number E-26/200.840/2021); Coordination of Higher Education Personnel Improvement - Finance Code 001; and by the Ministry of Science and Innovation (MICINN) through a National Project (PID2020-115454GB-C21).

Appendix A. Ablation study

In Section 3, we specified how to use the total rewards of each episode in order to calculate a tree agent's fitness (Equation (1)). This calculation involves three components: the *average reward*, the *standard deviation of the reward*, and the *tree size*. We argued that though the average reward is effectively what drives the success rate, including the standard deviation is key to producing trees with consistent behavior, while including tree size guides the process towards smaller, more interpretable trees. In this section, we provide an ablation study to justify the inclusion of these two components, showing that removing them from the equation results in worse solutions.

To achieve this goal, we modify the fitness calculation of the MENS-DT-RL to create four versions:

1. *Fitness A*: the full fitness, with standard deviation and tree size, as used in the paper:

$$fitness(M) = \frac{1}{N} \sum_{i=1}^N G_0^{(M,i)} - \sqrt{\frac{1}{N} \sum_{j=1}^N \left(G_0^{(M,j)} - \frac{1}{N} \sum_{i=1}^N G_0^{(M,i)} \right)^2} - \alpha ||M|| \quad (A.1)$$

2. *Fitness B*: the full fitness, but without standard deviation:

$$fitness(M) = \frac{1}{N} \sum_{i=1}^N G_0^{(M,i)} - \alpha ||M|| \quad (A.2)$$

3. *Fitness C*: the full fitness, but without tree size:

$$fitness(M) = \frac{1}{N} \sum_{i=1}^N G_0^{(M,i)} - \sqrt{\frac{1}{N} \sum_{j=1}^N \left(G_0^{(M,j)} - \frac{1}{N} \sum_{i=1}^N G_0^{(M,i)} \right)^2} \quad (A.3)$$

4. *Fitness D*: using only the average reward as fitness:

$$fitness(M) = \frac{1}{N} \sum_{i=1}^N G_0^{(M,i)} \quad (A.4)$$

In order to compare these four models, we focus on the Mountain Car environment, which strikes a good balance between simplicity and complexity (not as simple as Cartpole, but complex enough for there to be a reasonable difference between algorithms). The three different initializations of the MENS-DT-RL are executed for 50 simulations each, with the same parameterization as in the main text. The results are displayed in Table A.6.

As the results show, varying the fitness function can greatly affect the final solution: the configurations with smaller tree size are usually “A” and “B” (where tree size is part of the fitness function), while the configurations with lower standard deviation are usually “A” and “C” (where standard deviation is taken into account). To better see this effect, it is useful to depict the results visually. Fig. A.17 shows the boxplots of all four configurations for MENS-DT-RL (R), where it can be seen that, at the extremes, fitness A leads to solutions with lower tree size and standard deviation, while fitness D goes in the opposite direction. Interestingly, this has no clear effect on success rate, since this metric has such a large variance for each configuration; this is typical of random initializations like that of MENS-DT-RL (R), which can sometimes jumpstart the model with a slightly decent solution, and other times leave it stranded with a range of 0.00 success rate solutions that need to be completely modified by the genetic operators. Therefore, for the MENS-DT-RL (R), fitness A and B are clearly better than C and D (since they have similar success rates and lower

Table A.6
Results for the ablation study done for the Mountain Car task. Fitnesses C and D do not include tree size, while fitnesses B and D do not include standard deviation of reward.

Initialization	Config.	Avg. Reward	Avg. Stdev. Reward	Average SR	Avg. #Nodes
MENS-DT-RL (R)	Fitness A	-112.98	2.08	0.28	7.44
	Fitness B	-108.77	5.61	0.41	9.00
	Fitness C	-114.82	2.37	0.25	25.20
	Fitness D	-107.37	6.30	0.45	23.16
MENS-DT-RL (IL)	Fitness A	-106.30	4.49	0.74	7.00
	Fitness B	-103.98	11.14	0.40	7.20
	Fitness C	-103.53	4.95	0.54	25.48
	Fitness D	-101.76	9.27	0.81	21.84
MENS-DT-RL (P)	Fitness A	-106.68	5.94	0.74	7.12
	Fitness B	-106.87	10.32	0.38	6.40
	Fitness C	-105.78	3.12	0.76	22.44
	Fitness D	-104.36	8.73	0.59	19.80

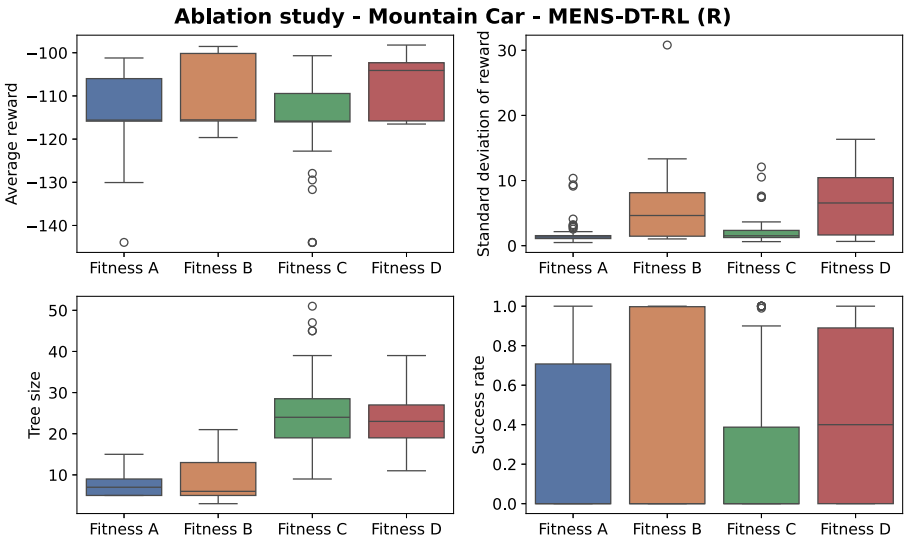


Fig. A.17. Ablation comparison between results produced by the MENS-DT-RL (R) when using different fitness functions on the Mountain Car task. Fitnesses C and D do not include tree size, while fitnesses B and D do not include standard deviation of reward.

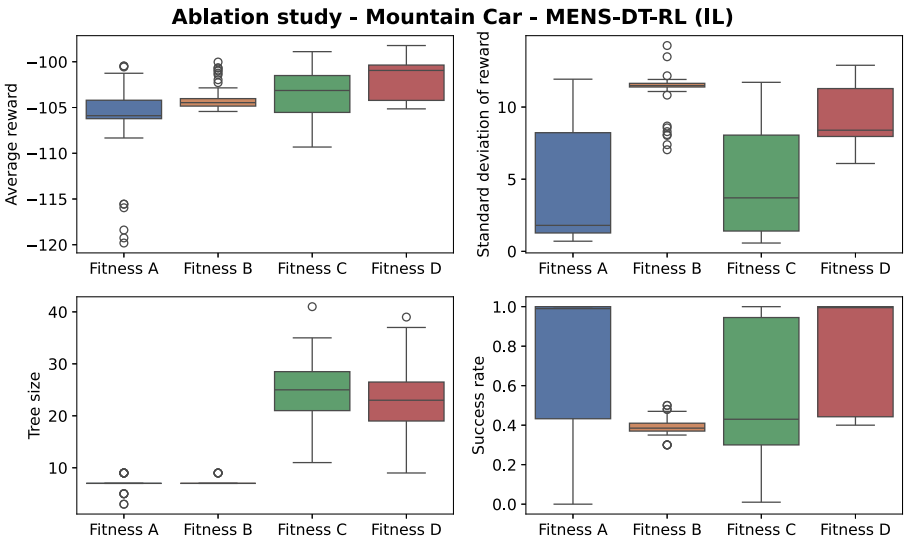


Fig. A.18. Ablation comparison between results produced by MENS-DT-RL (IL) when using different fitness functions on the Mountain Car task. Fitnesses C and D do not include tree size, while fitnesses B and D do not include standard deviation of reward.

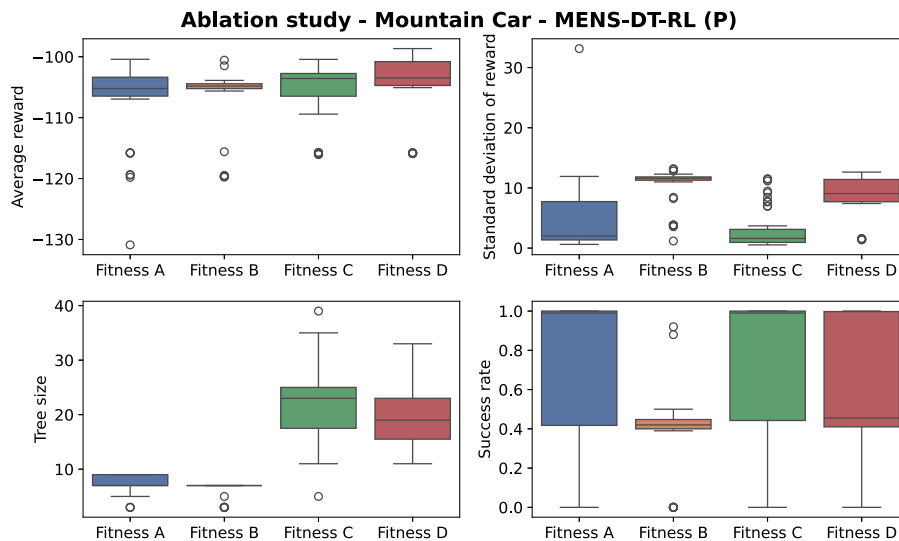


Fig. A.19. Ablation comparison between results produced by MENS-DT-RL (P) when using different fitness functions on the Mountain Car task. Fitnesses C and D do not include tree size, while fitnesses B and D do not include standard deviation of reward.

tree size), but A is not definitely better than B (A's standard deviation is lower, but this has no conclusive impact on success rate, given the high variance).

To see how including standard deviation drives success rate up, we must turn ourselves to MENS-DT-RL (IL), see Fig. A.18. As in the previous initialization, configurations “A” and “B” continue to stand out in terms of low tree size, while “A” and “C” do the same for standard deviation; however, there is also a marked difference in success rate that was absent in the previous initialization: now, fitness A has considerably higher success rate than fitness B, with little overlap between them. This serves as strong indication that including standard deviation in the fitness leads to models with higher success rates. The exact same behavior can be seen in Fig. A.19, which displays the boxplots for MENS-DT-RL (P). Overall, these results indicate that including both standard deviation and tree size in the fitness function leads to solutions with lower tree size and higher success rate.

References

- [1] I. Akkaya, M. Andrychowicz, M. Chociej, M. Litwin, B. McGrew, A. Petron, A. Paino, M. Plappert, G. Powell, R. Ribas, et al., Solving Rubik's cube with a robot hand, arXiv preprint, arXiv:1910.07113, 2019.
- [2] H. Mao, M. Alizadeh, I. Menache, S. Kandula, Resource management with deep reinforcement learning, in: Proceedings of the 15th ACM Workshop on Hot Topics in Networks, 2016, pp. 50–56.
- [3] G. Zheng, F. Zhang, Z. Zheng, Y. Xiang, N.J. Yuan, X. Xie, Z. Li, DRN: a deep reinforcement learning framework for news recommendation, in: Proceedings of the 2018 World Wide Web Conference, 2018, pp. 167–176.
- [4] D. Silver, A. Huang, C.J. Maddison, A. Guez, L. Sifre, G. Van Den Driessche, J. Schrittwieser, I. Antonoglou, V. Panneershelvam, M. Lanctot, et al., Mastering the game of go with deep neural networks and tree search, *Nature* 529 (7587) (2016) 484–489.
- [5] C. Rudin, Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead, *Nat. Mach. Intell.* 1 (5) (2019) 206–215.
- [6] G. Alain, Y. Bengio, Understanding intermediate layers using linear classifier probes, arXiv preprint, arXiv:1610.01644, 2016.
- [7] E.M. Kenny, C. Ford, M. Quinn, M.T. Keane, Explaining black-box classifiers using post-hoc explanations-by-example: the effect of explanations and error-rates in XAI user studies, *Artif. Intell.* 294 (2021) 103459.
- [8] C. Rudin, C. Chen, Z. Chen, H. Huang, L. Semenova, C. Zhong, Interpretable machine learning: fundamental principles and 10 grand challenges, *Stat. Surv.* 16 (2022) 1–85.
- [9] C. Glanois, P. Weng, M. Zimmer, D. Li, J. Hao, T. Yang, W. Liu, A survey on interpretable reinforcement learning, *IEEE Trans. Neural Netw. Learn. Syst.* (2021).
- [10] E. Puiutta, E. Veith, Explainable reinforcement learning: a survey, in: International Cross-Domain Conference for Machine Learning and Knowledge Extraction, Springer, 2020, pp. 77–95.
- [11] C. Yu, J. Liu, S. Nemati, G. Yin, Reinforcement learning in healthcare: a survey, *ACM Comput. Surv.* 55 (1) (2021) 1–36.
- [12] A. Coronato, M. Naeem, G. De Pietro, G. Paragliola, Reinforcement learning for intelligent healthcare applications: a survey, *Artif. Intell. Med.* 109 (2020) 101964.
- [13] B.R. Kiran, I. Sobh, V. Talpaert, P. Mannion, A.A. Al Sallab, S. Yogamani, P. Pérez, Deep reinforcement learning for autonomous driving: a survey, *IEEE Trans. Intell. Transp. Syst.* 23 (6) (2021) 4909–4926.
- [14] P. Sequeira, M. Gervasio, Interestingness elements for explainable reinforcement learning: understanding agents' capabilities and limitations, *Artif. Intell.* 288 (2020) 103367.
- [15] A. Alharin, T.-N. Doan, M. Sartipi, Reinforcement learning interpretation methods: a survey, *IEEE Access* 8 (2020) 171058–171077.
- [16] V.G. Costa, C.E. Pedreira, Recent advances in decision trees: an updated survey, *Artif. Intell. Rev.* (2022) 1–36.
- [17] O. Loyola-Gonzalez, Black-box vs. white-box: understanding their advantages and weaknesses from a practical point of view, *IEEE Access* 7 (2019) 154096–154113.
- [18] X. Meng, P. Zhang, Y. Xu, H. Xie, Construction of decision tree based on C4.5 algorithm for online voltage stability assessment, *Int. J. Electr. Power Energy Syst.* 118 (2020) 105793.
- [19] G. Ciravegna, P. Barbiero, F. Giannini, M. Gori, P. Lió, M. Maggini, S. Melacci, Logic explained networks, *Artif. Intell.* 314 (2023) 103822.

- [20] A. Silva, M. Gombolay, T. Killian, I. Jimenez, S.-H. Son, Optimization methods for interpretable differentiable decision trees applied to reinforcement learning, in: *International Conference on Artificial Intelligence and Statistics*, PMLR, 2020, pp. 1855–1865.
- [21] A.M. Roth, N. Topin, P. Jamshidi, M. Veloso, Conservative Q-improvement: reinforcement learning for an interpretable decision-tree policy, *arXiv preprint, arXiv:1907.01180*, 2019.
- [22] S. Salcedo-Sanz, C. Camacho-Gómez, D. Molina, F. Herrera, A coral reefs optimization algorithm with substrate layers and local search for large scale global optimization, in: *2016 IEEE Congress on Evolutionary Computation (CEC)*, IEEE, 2016, pp. 3574–3581.
- [23] S. Salcedo-Sanz, A review on the coral reefs optimization algorithm: new development lines and current applications, *Prog. Artif. Intell.* 6 (2017) 1–15.
- [24] Y.-C. Wang, C.-W. Tsai, An efficient coral reef optimization with substrate layers for clustering problem on spark, in: *2018 IEEE International Conference on Systems, Man, and Cybernetics (SMC)*, IEEE, 2018, pp. 2814–2819.
- [25] L. García-Hernández, J. García-Hernandez, L. Salas-Morera, C. Carmona-Muñoz, N.S. Alghamdi, J.V. de Oliveira, S. Salcedo-Sanz, Addressing unequal area facility layout problems with the coral reef optimization algorithm with substrate layers, *Eng. Appl. Artif. Intell.* 93 (2020) 103697.
- [26] C. Marcelino, J. Pérez-Aracil, E. Wanner, S. Jiménez-Fernández, G. Leite, S. Salcedo-Sanz, Cross-entropy boosted CRO-SL for optimal power flow in smart grids, *Soft Comput.* 27 (10) (2023) 6549–6572.
- [27] E. Bermejo, M. Chica, S. Damas, S. Salcedo-Sanz, O. Cordón, Coral reef optimization with substrate layers for medical image registration, *Swarm Evol. Comput.* 42 (2018) 138–159.
- [28] G. Brockman, V. Cheung, L. Pettersson, J. Schneider, J. Schulman, J. Tang, W. Zaremba, OpenAI gym, *arXiv preprint, arXiv:1606.01540*, 2016.
- [29] R. Gautron, E.J. Padrón, P. Preux, J. Bigot, O.-A. Maillard, D. Emukpere, gym-DSSAT: a crop model turned into a reinforcement learning environment, Ph.D. thesis, Inria Lille, 2022.
- [30] R.S. Sutton, A.G. Barto, *Reinforcement Learning: An Introduction*, MIT Press, 2018.
- [31] C.J. Watkins, P. Dayan, Q-learning, *Mach. Learn.* 8 (1992) 279–292.
- [32] P.Y. Glorennec, L. Joffe, Fuzzy q-learning, in: *Proceedings of 6th International Fuzzy Systems Conference*, vol. 2, IEEE, 1997, pp. 659–662.
- [33] H. Hasselt, Double q-learning, *Adv. Neural Inf. Process. Syst.* 23 (2010).
- [34] V. Mnih, K. Kavukcuoglu, D. Silver, A.A. Rusu, J. Veness, M.G. Bellemare, A. Graves, M. Riedmiller, A.K. Fidjeland, G. Ostrovski, et al., Human-level control through deep reinforcement learning, *Nature* 518 (7540) (2015) 529–533.
- [35] A. Hussein, M.M. Gaber, E. Elyan, C. Jayne, Imitation learning: a survey of learning methods, *ACM Comput. Surv.* 50 (2) (2017) 1–35.
- [36] M. Bojarski, D. Del Testa, D. Dworakowski, B. Firner, B. Flepp, P. Goyal, L.D. Jackel, M. Monfort, U. Muller, J. Zhang, et al., End to end learning for self-driving cars, *arXiv preprint, arXiv:1604.07316*, 2016.
- [37] J. Merel, Y. Tassa, D. TB, S. Srinivasan, J. Lemmon, Z. Wang, G. Wayne, N. Heess, Learning human behaviors from motion capture by adversarial imitation, *arXiv preprint, arXiv:1707.02201*, 2017.
- [38] C. Finn, S. Levine, P. Abbeel, Guided cost learning: deep inverse optimal control via policy optimization, in: *International Conference on Machine Learning*, PMLR, 2016, pp. 49–58.
- [39] D.A. Pomerleau, ALVINN: an autonomous land vehicle in a neural network, *Adv. Neural Inf. Process. Syst.* 1 (1988).
- [40] S. Ross, G. Gordon, D. Bagnell, A reduction of imitation learning and structured prediction to no-regret online learning, in: *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics, JMLR Workshop and Conference Proceedings*, 2011, pp. 627–635.
- [41] G. Wu, R. Mallipeddi, P.N. Suganthan, Ensemble strategies for population-based optimization algorithms—a survey, *Swarm Evol. Comput.* 44 (2019) 695–711.
- [42] J.A. Vrugt, B.A. Robinson, Improved evolutionary optimization from genetically adaptive multimethod search, *Proc. Natl. Acad. Sci.* 104 (3) (2007) 708–711.
- [43] J.A. Vrugt, B.A. Robinson, J.M. Hyman, Self-adaptive multimethod search for global optimization in real-parameter spaces, *IEEE Trans. Evol. Comput.* 13 (2) (2008) 243–259.
- [44] W.K. Mashwani, A. Salhi, Multiobjective evolutionary algorithm based on multimethod with dynamic resources allocation, *Appl. Soft Comput.* 39 (2016) 292–309.
- [45] Y. Xue, S. Zhong, Y. Zhuang, B. Xu, An ensemble algorithm with self-adaptive learning techniques for high-dimensional numerical optimization, *Appl. Math. Comput.* 231 (2014) 329–346.
- [46] X. Wang, C. Li, J. Zhu, Q. Meng, L-SHADE-E: ensemble of two differential evolution algorithms originating from L-SHADE, *Inf. Sci.* 552 (2021) 201–219.
- [47] J. Yao, Z. Chen, Z. Liu, Improved ensemble of differential evolution variants, *PLoS ONE* 16 (8) (2021) e0256206.
- [48] S. Salcedo-Sanz, R. García-Herrera, C. Camacho-Gómez, E. Alexandre, L. Carro-Calvo, F. Jaume-Santero, Near-optimal selection of representative measuring points for robust temperature field reconstruction with the CRO-SL and analogue methods, *Glob. Planet. Change* 178 (2019) 15–34.
- [49] J. Pérez-Aracil, C. Camacho-Gómez, A.M. Hernández-Díaz, E. Pereira, D. Camacho, S. Salcedo-Sanz, Memetic coral reefs optimization algorithms for optimal geometrical design of submerged arches, *Swarm Evol. Comput.* 67 (2021) 100958.
- [50] J. Pérez-Aracil, D. Casillas-Pérez, S. Jiménez-Fernández, L. Prieto-Godino, S. Salcedo-Sanz, A versatile multi-method ensemble for wind farm layout optimization, *J. Wind Eng. Ind. Aerodyn.* 225 (2022) 104991.
- [51] J. Pérez-Aracil, C. Camacho-Gómez, E. Lorente-Ramos, C.M. Marina, L.M. Cornejo-Bueno, S. Salcedo-Sanz, New probabilistic, dynamic multi-method ensembles for optimization based on the CRO-SL, *Mathematics* 11 (7) (2023) 1666.
- [52] D. Hein, S. Udluft, T.A. Runkler, Interpretable policies for reinforcement learning by genetic programming, *Eng. Appl. Artif. Intell.* 76 (2018) 158–169.
- [53] H. Zhang, A. Zhou, X. Lin, Interpretable policy derivation for reinforcement learning based on evolutionary feature synthesis, *Complex Intell. Syst.* 6 (2020) 741–753.
- [54] D. Trivedi, J. Zhang, S.-H. Sun, J.J. Lim, Learning to synthesize programs as interpretable and generalizable policies, *Adv. Neural Inf. Process. Syst.* 34 (2021) 25146–25163.
- [55] A. Verma, V. Murali, R. Singh, P. Kohli, S. Chaudhuri, Programmatically interpretable reinforcement learning, in: *International Conference on Machine Learning*, PMLR, 2018, pp. 5045–5054.
- [56] T. Silver, K.R. Allen, A.K. Lew, L.P. Kaelbling, J. Tenenbaum, Few-shot Bayesian imitation learning with logical program policies, in: *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34, 2020, pp. 10251–10258.
- [57] E.H. Mamdani, Application of fuzzy algorithms for control of simple dynamic plant, in: *Proceedings of the Institution of Electrical Engineers*, vol. 121, IET, 1974, pp. 1585–1588.
- [58] C.-F. Juang, J.-Y. Lin, C.-T. Lin, Genetic reinforcement learning through symbiotic evolution for fuzzy controller design, *IEEE Trans. Syst. Man Cybern., Part B, Cybern.* 30 (2) (2000) 290–302.
- [59] J. Huang, P.P. Angelov, C. Yin, Interpretable policies for reinforcement learning by empirical fuzzy sets, *Eng. Appl. Artif. Intell.* 91 (2020) 103559.
- [60] D. Hein, A. Hentschel, T. Runkler, S. Udluft, Particle Swarm Optimization for generating interpretable fuzzy reinforcement learning policies, *Eng. Appl. Artif. Intell.* 65 (2017) 87–98.
- [61] L. Breiman, J.H. Friedman, R.A. Olshen, C.J. Stone, *Classification and Regression Trees*, Routledge, 2017.
- [62] J.R. Quinlan, *C4.5: Programs for Machine Learning*, Elsevier, 2014.
- [63] L.D. Pyeatt, A.E. Howe, et al., Decision tree function approximation in reinforcement learning, in: *Proceedings of the Third International Symposium on Adaptive Systems: Evolutionary Computation and Probabilistic Graphical Models*, Cuba, vol. 2, 2001, pp. 70–77.
- [64] A.K. McCallum, *Reinforcement Learning with Selective Perception and Hidden State*, University of Rochester, 1996.
- [65] W.T. Uther, M.M. Veloso, Tree based discretization for continuous state space reinforcement learning, in: *AAAI/IAAI*, vol. 98, 1998, pp. 769–774.

- [66] D. Ernst, P. Geurts, L. Wehenkel, Tree-based batch mode reinforcement learning, *J. Mach. Learn. Res.* 6 (2005).
- [67] O. Bastani, Y. Pu, A. Solar-Lezama, Verifiable reinforcement learning via policy extraction, *Adv. Neural Inf. Process. Syst.* 31 (2018).
- [68] Y. Coppens, K. Efthymiadis, T. Lenaerts, A. Nowé, T. Miller, R. Weber, D. Magazzini, Distilling deep reinforcement learning policies in soft decision trees, in: *Proceedings of the IJCAI 2019 Workshop on Explainable Artificial Intelligence*, 2019, pp. 1–6.
- [69] N. Frosst, G. Hinton, Distilling a neural network into a soft decision tree, *arXiv preprint, arXiv:1711.09784*, 2017.
- [70] H. Li, J. Song, M. Xue, H. Zhang, J. Ye, L. Cheng, M. Song, A survey of neural trees, *arXiv preprint, arXiv:2209.03415*, 2022.
- [71] G. Liu, O. Schulte, W. Zhu, Q. Li, Toward interpretable deep reinforcement learning with linear model U-trees, in: *Joint European Conference on Machine Learning and Knowledge Discovery in Databases*, Springer, 2019, pp. 414–429.
- [72] A. Suárez, J.F. Lutsko, Globally optimal fuzzy decision trees for classification and regression, *IEEE Trans. Pattern Anal. Mach. Intell.* 21 (12) (1999) 1297–1311.
- [73] L.L. Custode, G. Iacca, Evolutionary learning of interpretable decision trees, *IEEE Access* 11 (2023) 6169–6184.
- [74] Y. Dhebar, K. Deb, S. Nagesh Rao, L. Zhu, D. Filev, Towards interpretable-AI policies induction using evolutionary nonlinear decision trees for discrete action systems, *arXiv preprint, arXiv:2009.09521*, 2020.
- [75] R.A. Lopes, A. Freitas, R. Silva, F.G. Guimarães, Differential evolution and perceptron decision trees for classification tasks, in: *International Conference on Intelligent Data Engineering and Automated Learning*, Springer, 2012, pp. 550–557.
- [76] R. Rivera-Lopez, J. Canul-Reich, A global search approach for inducing oblique decision trees using differential evolution, in: *Canadian Conference on Artificial Intelligence*, Springer, 2017, pp. 27–38.
- [77] V.G. Costa, S. Salcedo-Sanz, C.E. Pedreira, Efficient evolution of decision trees via fully matrix-based fitness evaluation, *Appl. Soft Comput.* (2023) 111045.
- [78] A.A. Freitas, Comprehensive classification models: a position paper, *ACM SIGKDD Explor. Newsl.* 15 (1) (2014) 1–10.
- [79] B. Kazimipour, X. Li, A.K. Qin, A review of population initialization techniques for evolutionary algorithms, in: *2014 IEEE Congress on Evolutionary Computation (CEC)*, IEEE, 2014, pp. 2585–2592.
- [80] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, N. Dormann, Stable-Baselines3: reliable reinforcement learning implementations, *J. Mach. Learn. Res.* 22 (1) (2021) 12348–12355.
- [81] A.W. Moore, Efficient memory-based learning for robot control, *Tech. Rep.*, University of Cambridge, 1990.
- [82] J.W. Jones, G. Hoogenboom, C.H. Porter, K.J. Boote, W.D. Batchelor, L. Hunt, P.W. Wilkens, U. Singh, A.J. Gijssman, J.T. Ritchie, The DSSAT cropping system model, *Eur. J. Agron.* 18 (3–4) (2003) 235–265.
- [83] G. Hoogenboom, C.H. Porter, K.J. Boote, V. Shelia, P.W. Wilkens, U. Singh, J.W. White, S. Asseng, J.I. Lizaso, L.P. Moreno, et al., The DSSAT crop modeling ecosystem, in: *Advances in Crop Modelling for a Sustainable Agriculture*, Burleigh Dodds Science Publishing, 2019, pp. 173–216.
- [84] L.L. Custode, G. Iacca, Evolutionary learning of interpretable decision trees, *arXiv preprint, arXiv:2012.07723*, 2020.
- [85] L. Hunt, K. Boote, Data for model operation, calibration, and evaluation, in: *Understanding Options for Agricultural Production*, 1998, pp. 9–39.
- [86] E. Mosqueira-Rey, E. Hernández-Pereira, D. Alonso-Ríos, J. Bobes-Bascarán, Á. Fernández-Leal, Human-in-the-loop machine learning: a state of the art, *Artif. Intell. Rev.* 56 (4) (2023) 3005–3054.